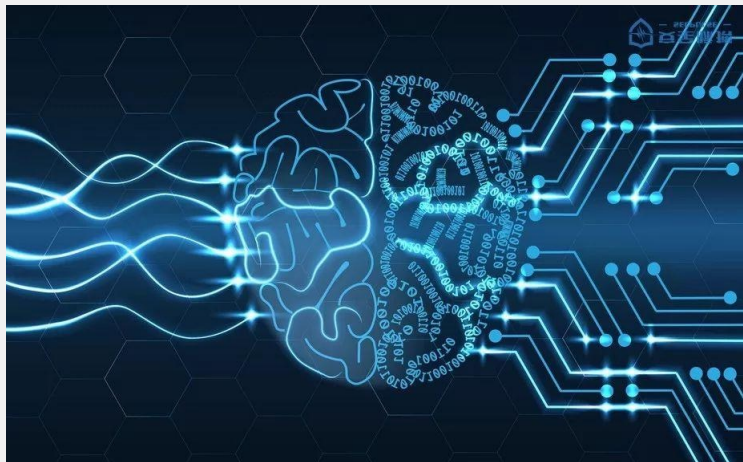




机器学习

Machine Learning

授课老师：程骋

办公室：南一楼中315

邮箱：c_cheng@hust.edu.cn

第六章：卷积神经网络

目录 CONTENTS

01 全连接神经网络在处理图像数据的问题

02 卷积神经网络工作原理

03 卷积神经网络结构

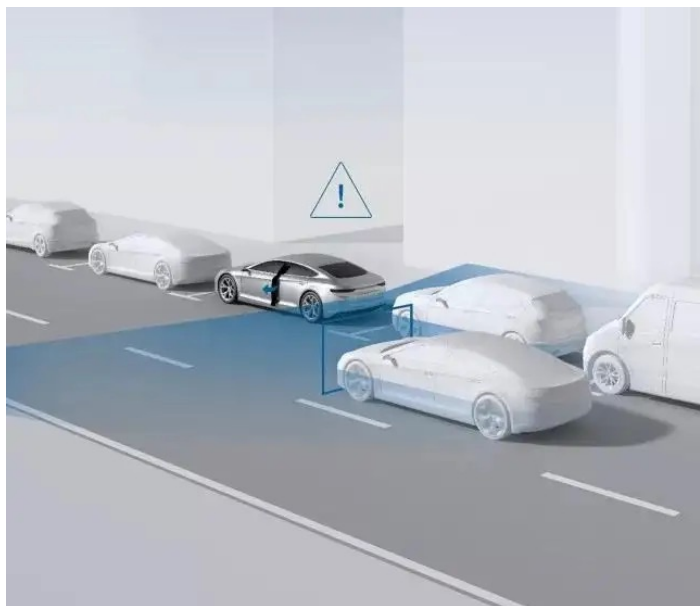
04 卷积神经网络的反向传播 为人师表

05 卷积神经网络的训练

06 LeNet/AlexNet/VGG/ResNet

□ 让机器理解**图片**或者视频中的内容

- 任务：图像分类、目标检测、图像分割
- 应用场景：医疗影像诊断、文字识别、视频监控、辅助驾驶等

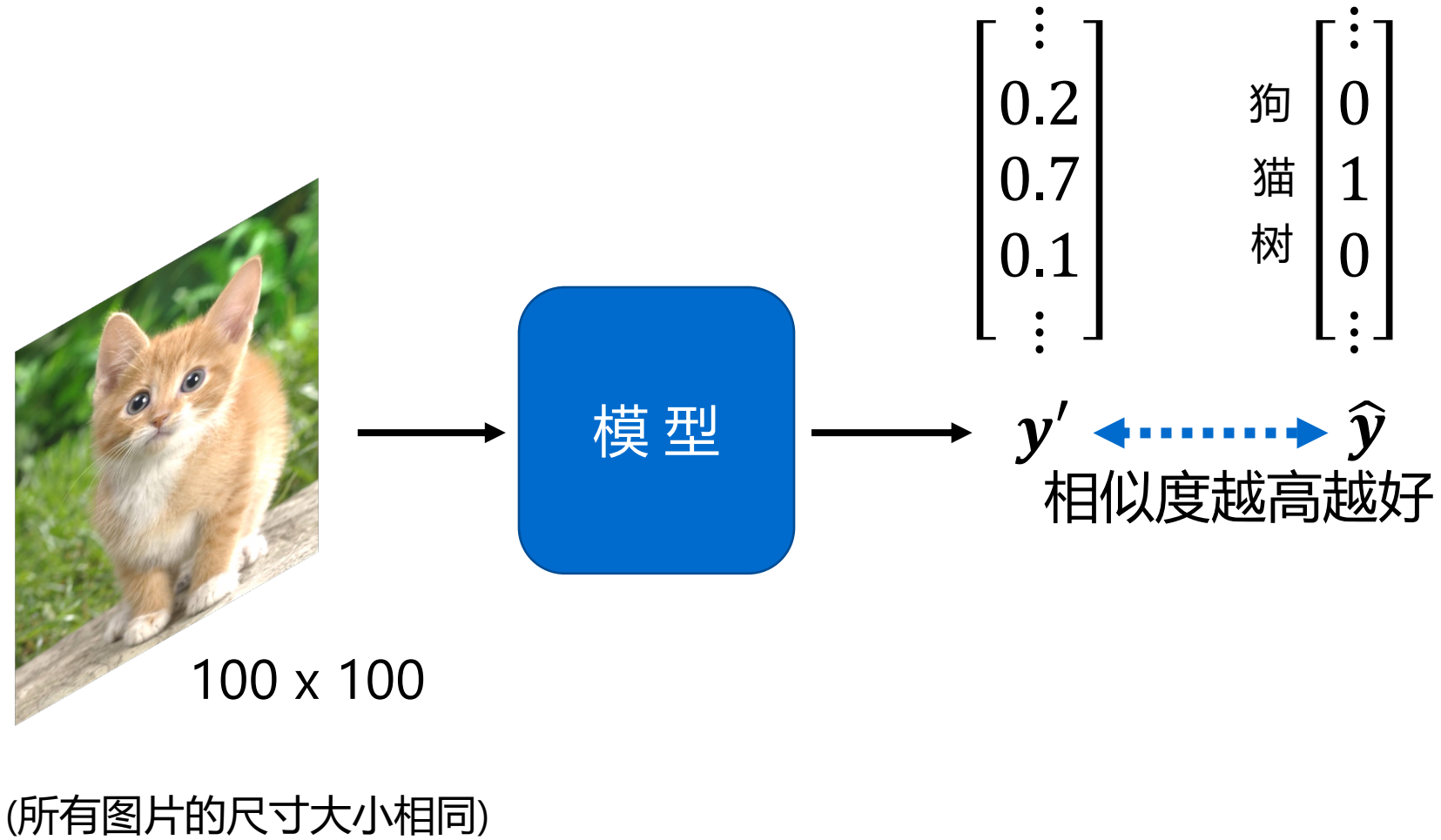


无人车对周围环境（行人、汽车等）的识别

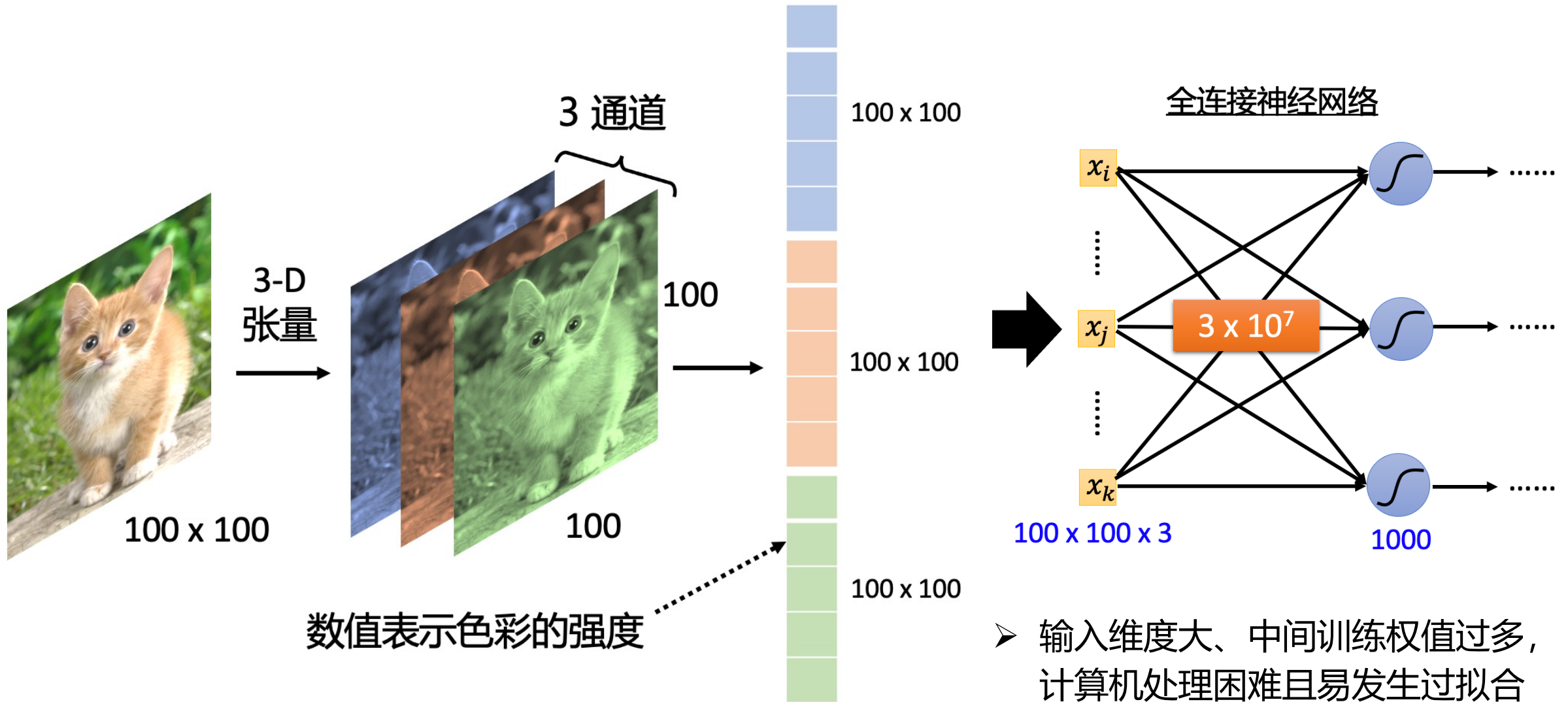
人脸识别

图像检索

1.全连接神经网络在处理图像数据的问题



1. 全连接神经网络在处理图像数据的问题

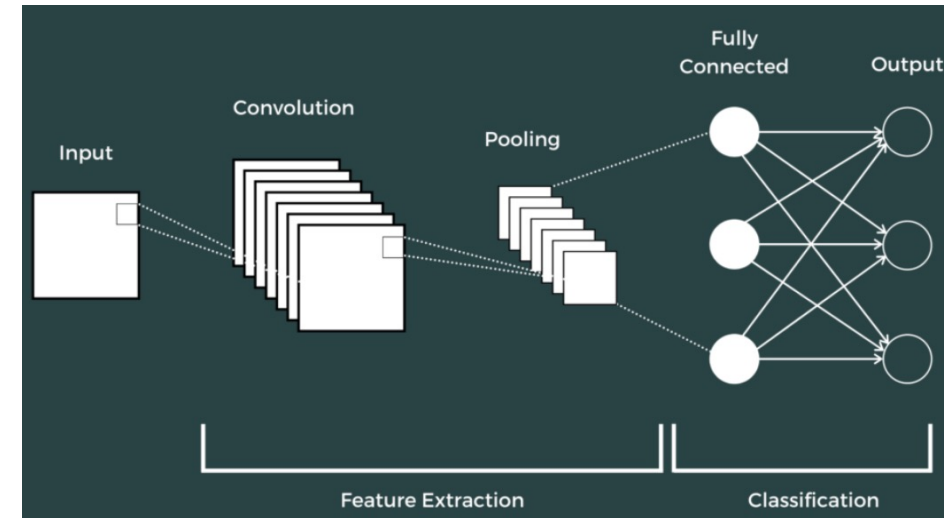
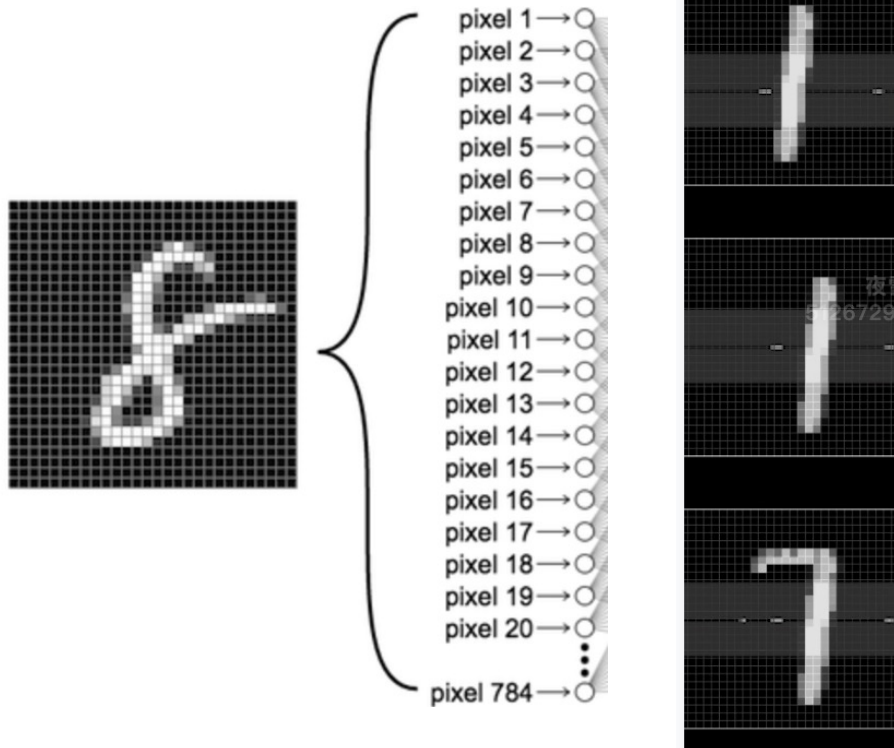


1.全连接神经网络在处理图像数据的问题

➤ 图像平铺成向量丢失掉了像素点上的空间联系

➤ 对图像移位、缩放和其他形式的变化不敏感

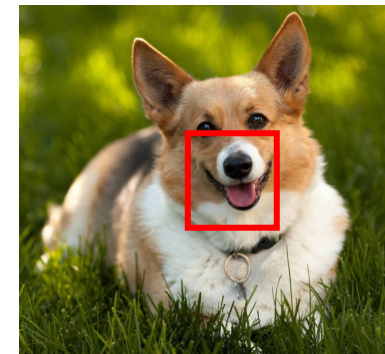
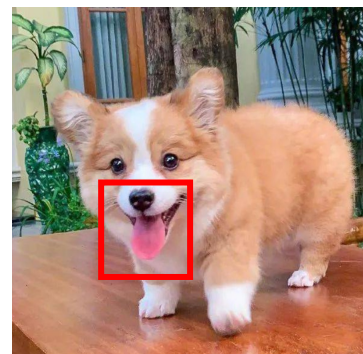
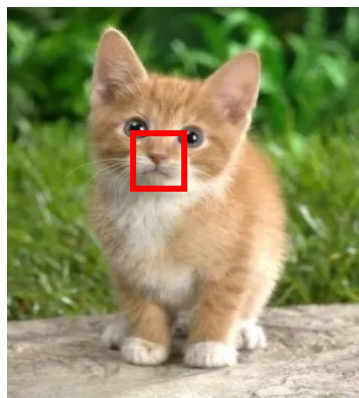
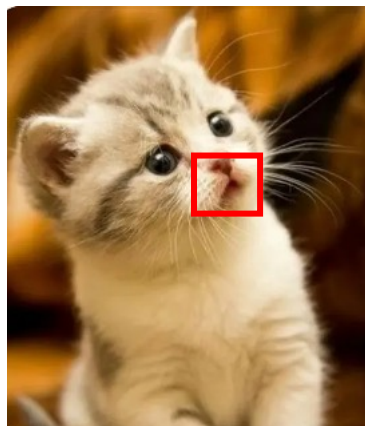
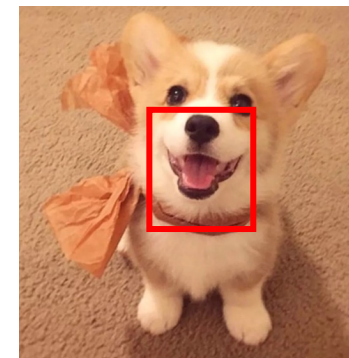
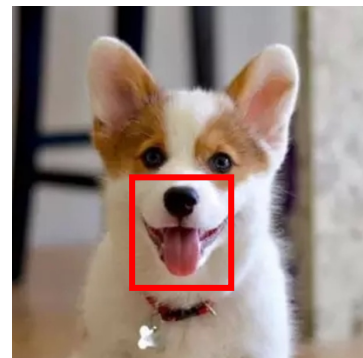
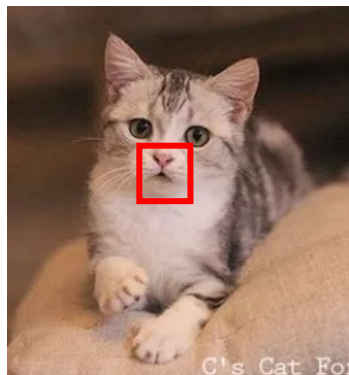
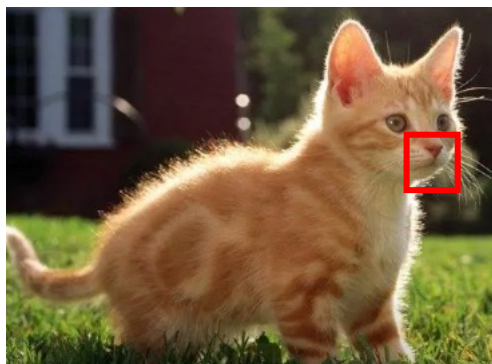
□ 解决方法:



卷积神经网络

2.卷积神经网络的工作原理

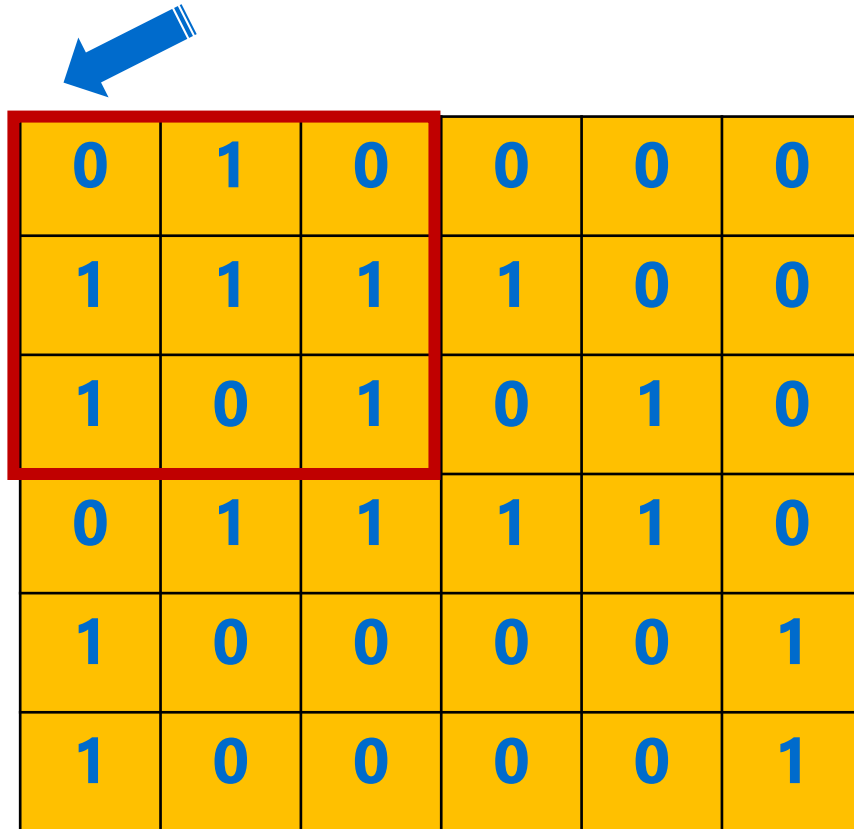
□ 猫狗分类



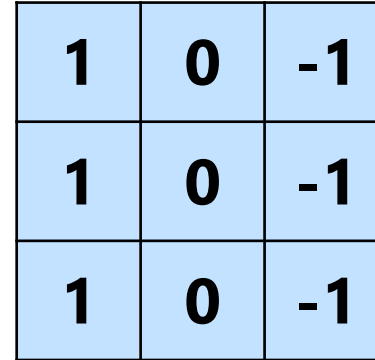
无需比对整个图片，提取局部特征

2.卷积神经网络的工作原理

Receptive field(感受野)



0	1	0	0	0	0
1	1	1	1	0	0
1	0	1	0	1	0
0	1	1	1	1	0
1	0	0	0	0	1
1	0	0	0	0	1



1	0	-1
1	0	-1
1	0	-1

Kernel(核)/Filter(过滤器)

Kernel size: 3*3

Stride(步幅):每次滑动的行数和列数称为stride。

二维卷积运算：将输入域卷积核做**互相关运算**，从最左上方开始，按从左往右、从上往下的顺序，二维核窗口一次在输入数组上滑动，当窗口滑动到某一位置时，**窗口中的输入数组与核数组按元素乘积并求和**，得到输出数组中相应位置的元素。

2. 卷积神经网络的工作原理

➤ Stride: 1, Kernel size: 3*3

0	1	0	0	0	0
1	1	1	1	0	0
1	0	1	0	1	0
0	1	1	1	1	0
1	0	0	0	0	1
1	0	0	0	0	1

1	0	-1
1	0	-1
1	0	-1

二维卷积



0			

2.卷积神经网络的工作原理

➤ Stride: 1, Kernel size: 3*3

0	1	0	0	0	0
1	1	1	1	0	0
1	0	1	0	1	0
0	1	1	1	1	0
1	0	0	0	0	1
1	0	0	0	0	1

1	0	-1
1	0	-1
1	0	-1

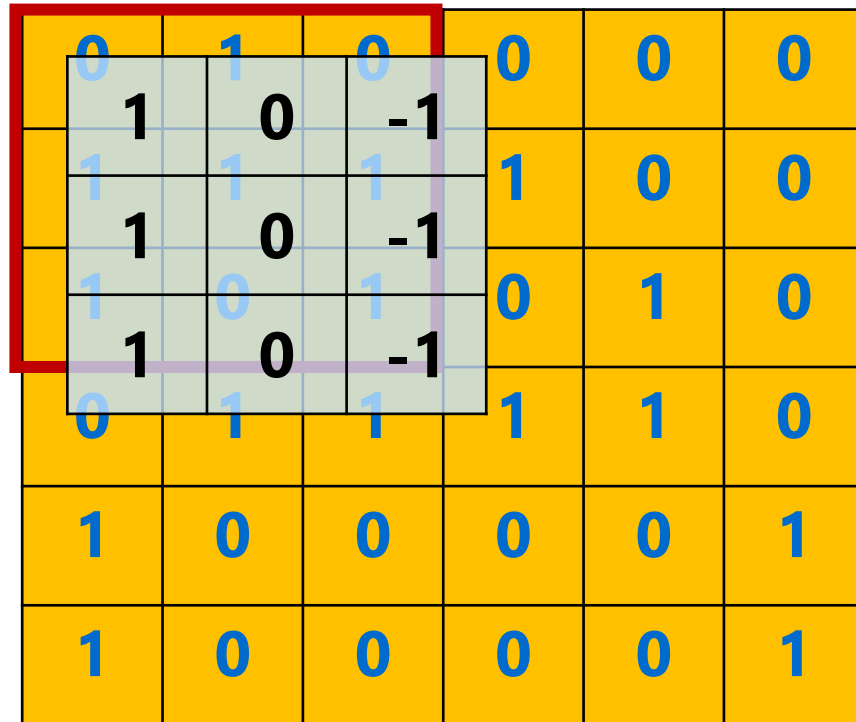
二维卷积



0	1		

2. 卷积神经网络的工作原理

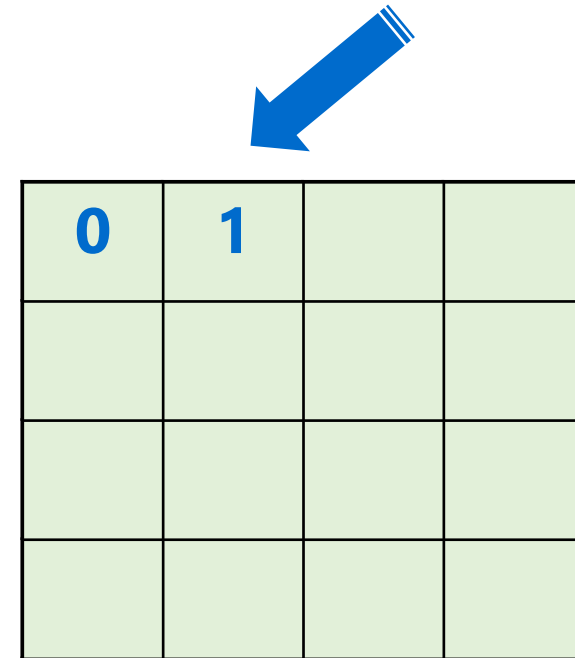
➤ Stride: 1, Kernel size: 3*3



二维卷积



Feature map (特征图)



假设输入图像的尺寸是 $n_h \times n_w$, 卷积核的大小是 $k_h \times k_w$, 输出的大小为:
 $(n_h - k_h + 1) \times (n_w - k_w + 1)$

2.卷积神经网络的工作原理

垂直边缘检测

1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0

Kernel

1	0	-1
1	0	-1
1	0	-1

0	0	3	3	0	0
0	0	3	3	0	0
0	0	3	3	0	0
0	0	3	3	0	0



2. 卷积神经网络的工作原理

➤ Stride: 2?, Kernel size: 3*3

0	1	0	0	1	0	-1
1	1	1	1	1	0	-1
1	0	1	0	1	0	-1
0	1	1	1	1	0	
1	0	0	0	0	1	
1	0	0	0	0	1	

Padding



0	1	0	0	0	0	0
1	1	1	1	0	0	0
1	0	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	1	0
1	0	0	0	0	1	0

Padding (填充)：在输入高和宽的两侧填充元素（ $p_w \times p_h$ ）（通常是0元素）。为了方便推测每个层的输出形状。对于stride=1，通常会设置 $p_h = k_h - 1$ 和 $p_w = k_w - 1$ 来使输入和输出具有相同的大小。

2. 卷积神经网络的工作原理

➤ Stride: 2?, Kernel size: 3*3

0	1	0	0	0	0	0
1	1	1	1	0	0	0
1	0	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	1	0
1	0	0	0	0	1	0
0	0	0	0	0	0	0

1	0	-1
1	0	-1
1	0	-1



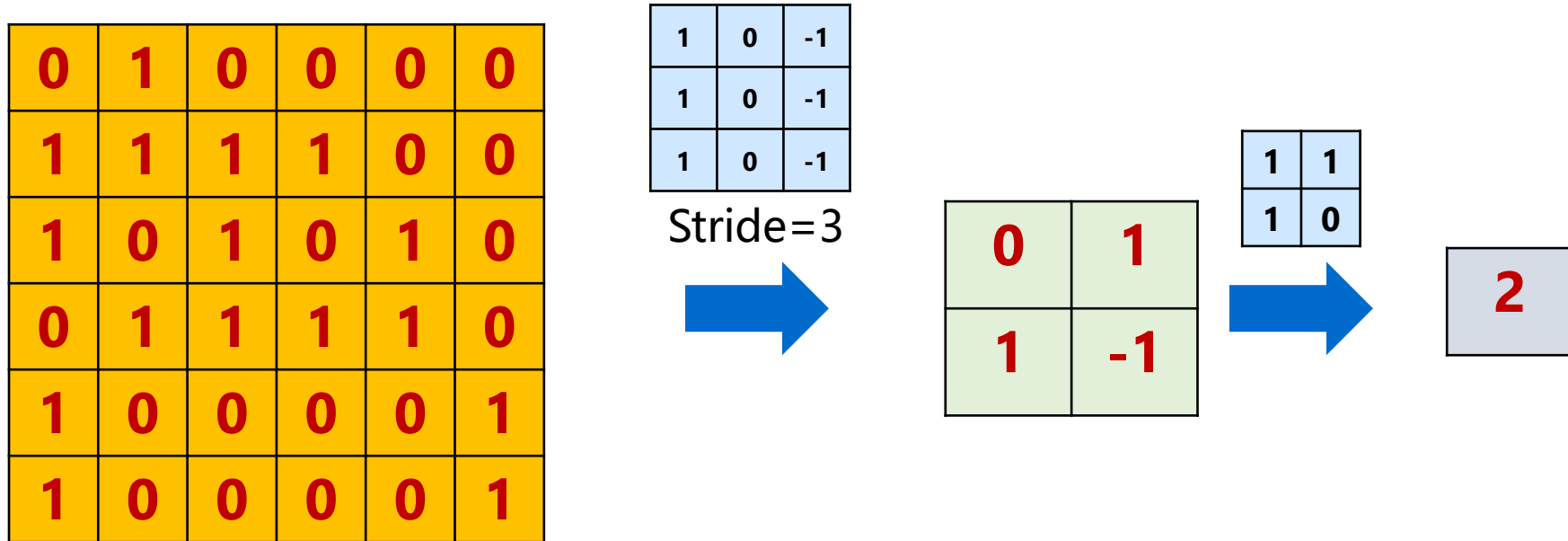
0	1	1
0	0	2
2	0	0

假设输入图像的尺寸是 $n_h \times n_w$, 卷积核的大小是 $k_h \times k_w$, padding 大小为 $p_w \times p_h$, 输出的大小为: $\left(\frac{n_h - k_h + p_h}{\text{stride}} + 1\right) \times \left(\frac{n_w - k_w + p_w}{\text{stride}} + 1\right)$

2.卷积神经网络的工作原理

局部连接：感受野

让每个卷积核只与输入数据的一个局部区域连接（感受野），会大大减少网络的**参数**，同时可以提取到局部特征。



通过更深的卷积神经网络使特征图中单个元素的感受野变的更加广阔，从而捕捉输入上更大尺寸的特征。

2.卷积神经网络的工作原理



➤ 测试:

假设输入的图像的尺寸是 $n_h \times n_w$, 卷积核的大小是 $k_h \times k_w$, 填充 (Padding) 的行为 p_h , 宽为 p_w , stride: 1, 输出的feature map的尺寸是多少:

$$(n_h - k_h + p_h + 1) \times (n_w - k_w + p_w + 1)$$

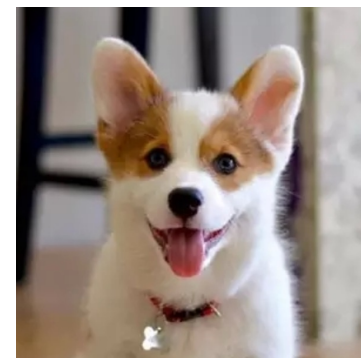
2.卷积神经网络的工作原理



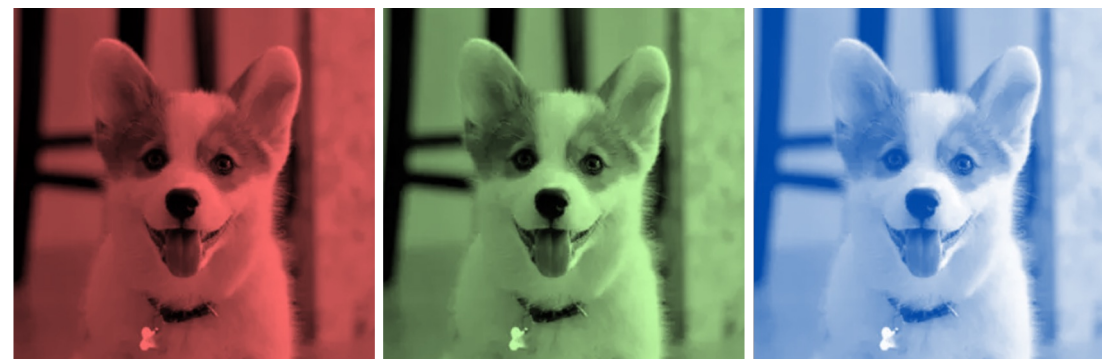
➤ 通道 (Channel)

0	1	0	0	0	0
1	1	1	1	0	0
1	0	1	0	1	0
0	1	1	1	1	0
1	0	0	0	0	1
1	0	0	0	0	1

Channel = 1



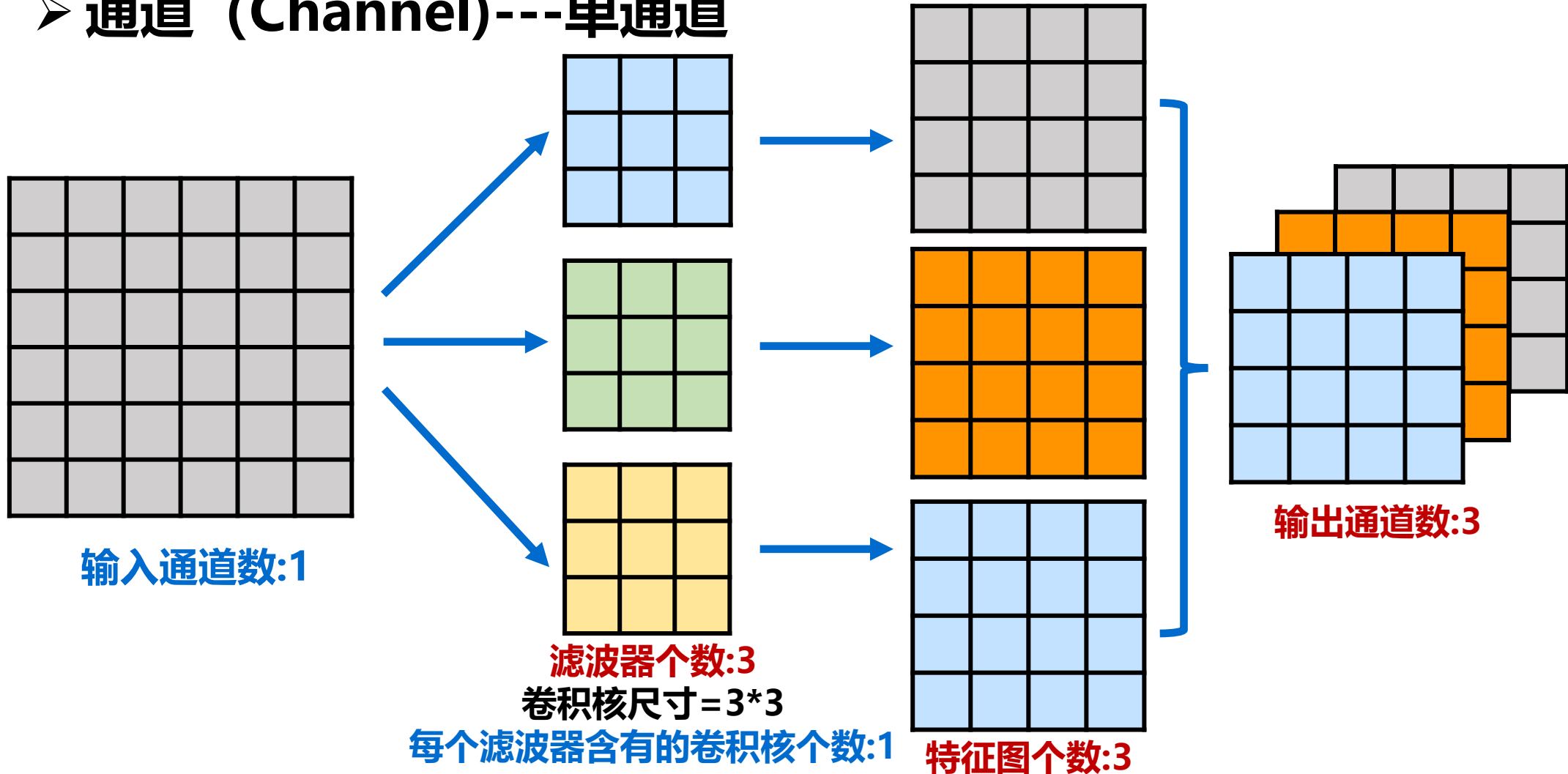
RGB类型



Channel = 3

2.卷积神经网络的工作原理

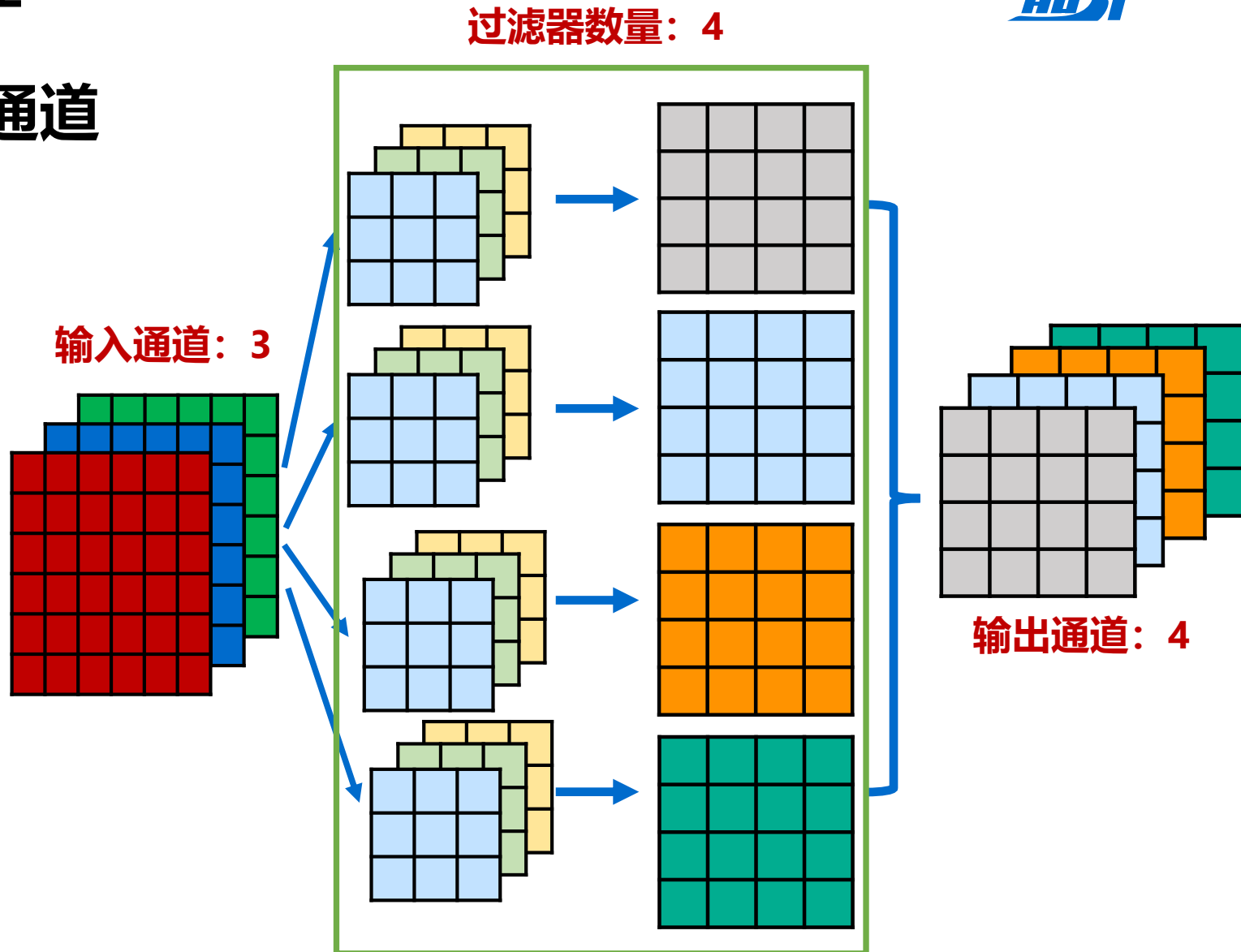
➤ 通道 (Channel)---单通道



2.卷积神经网络的工作原理

➤ 通道 (Channel)---多通道

- 对于最初输入图片样本的通道数取决于**图片的类型**，**如果是彩色的，通道数为3**，如果是灰度图，通道数为1
- 卷积完成之后，输出的通道数取决于**滤波器 (Filter)的数量**
- 对于第二层或者更多层的卷积，此时的输入通道数就是上一层输出通道数，这一层的输出通道数还是取决于过滤器数目

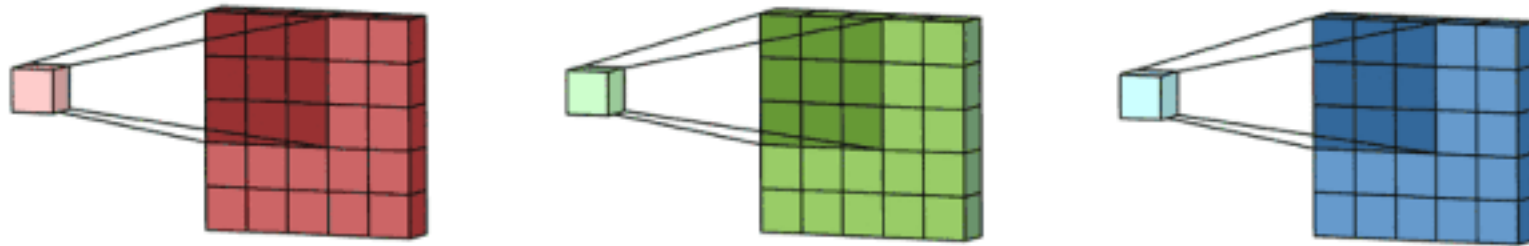


2.卷积神经网络的工作原理



- **通道 (Channel)**---多通道卷积：每个通道跟一个卷积核做卷积运算，然后将结果相加得到一个特征图的输出

第一步：

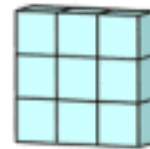
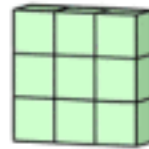
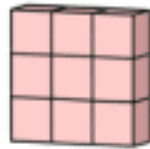


2.卷积神经网络的工作原理



- **通道 (Channel)**---多通道卷积：每个通道跟一个卷积核做卷积运算，然后将结果相加得到一个特征图的输出

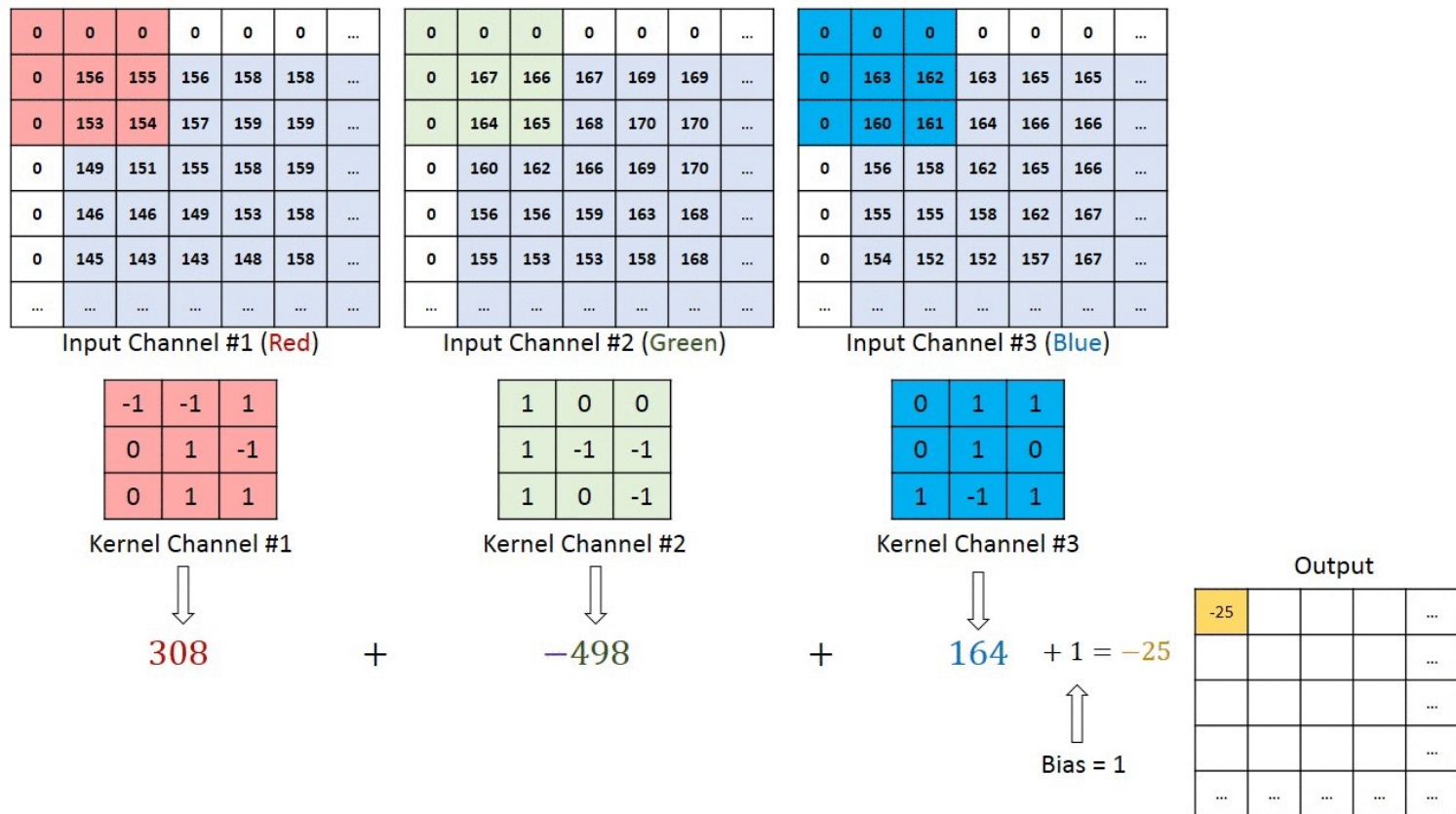
第二步：



2.卷积神经网络的工作原理



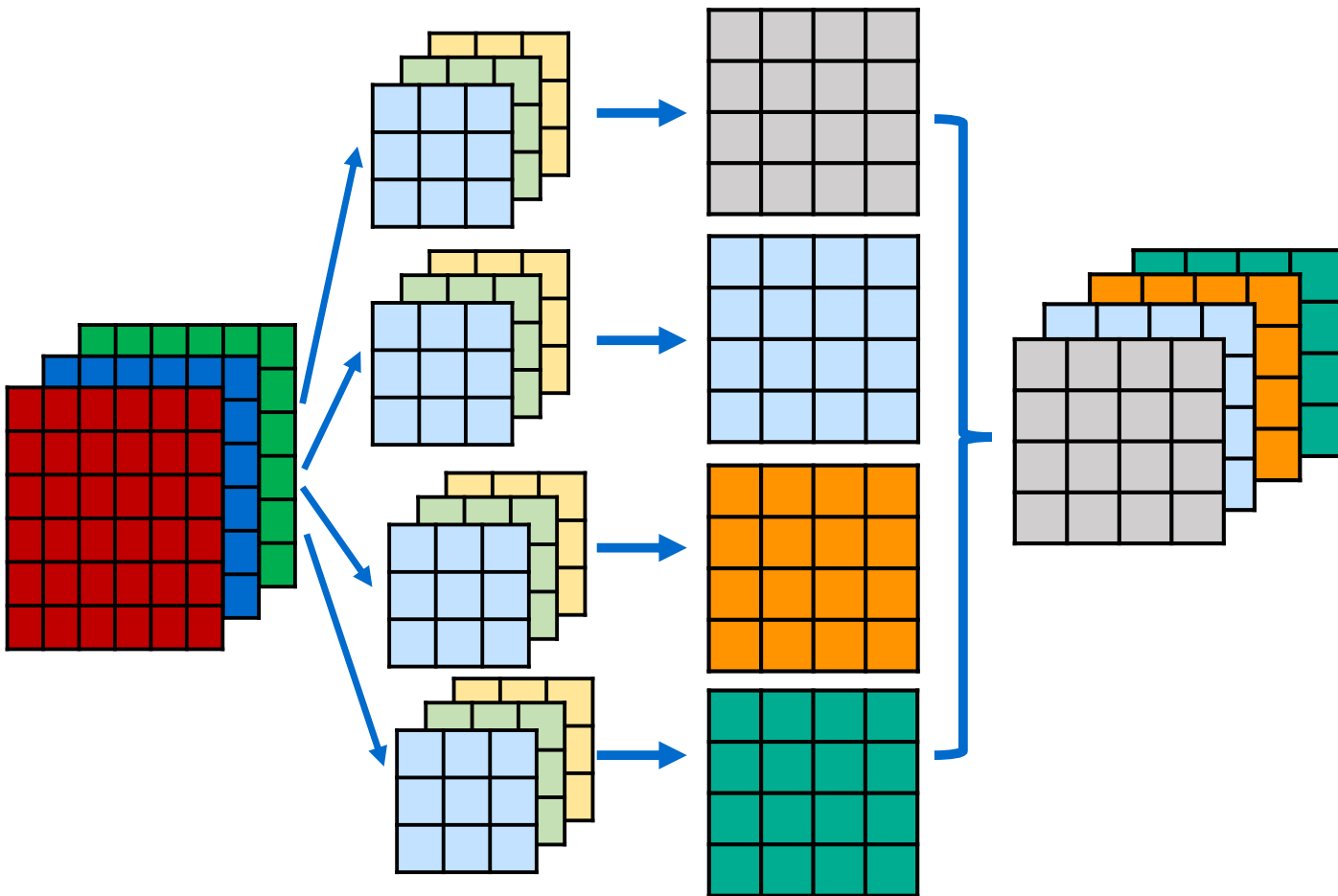
- **通道 (Channel)**---多通道卷积：每个通道跟一个卷积核做卷积运算，然后将结果相加得到一个特征图的输出



2.卷积神经网络的工作原理



➤ 通道 (Channel)---多通道



输入通道数: 3

滤波器数量: 4

卷积核尺寸: 3×3

每个滤波器含有的卷积核个数: 3

特征图个数: 4

输出通道个数: 4

2.卷积神经网络的工作原理

➤ 卷积层需要训练的参数

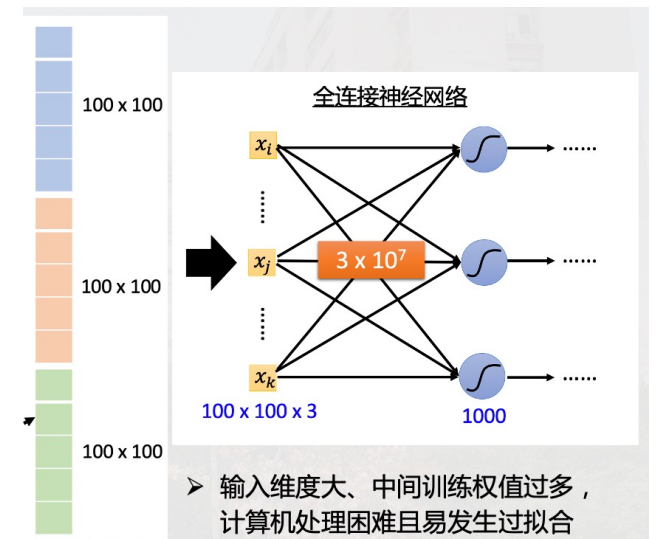
- ✓ 由卷积核的个数和大小决定
- ✓ 与图片的大小没关系

例：假设输入的图像的尺寸是 $100 \times 100 \times 3$, 卷积层使用10个 $3 \times 3 \times 3$ 的卷积核, 这一层需要训练的参数是多少？

$$10 \times (3 \times 3 \times 3 + 1) = 280$$

偏置

□ 相较于全连接神经网络，参数少

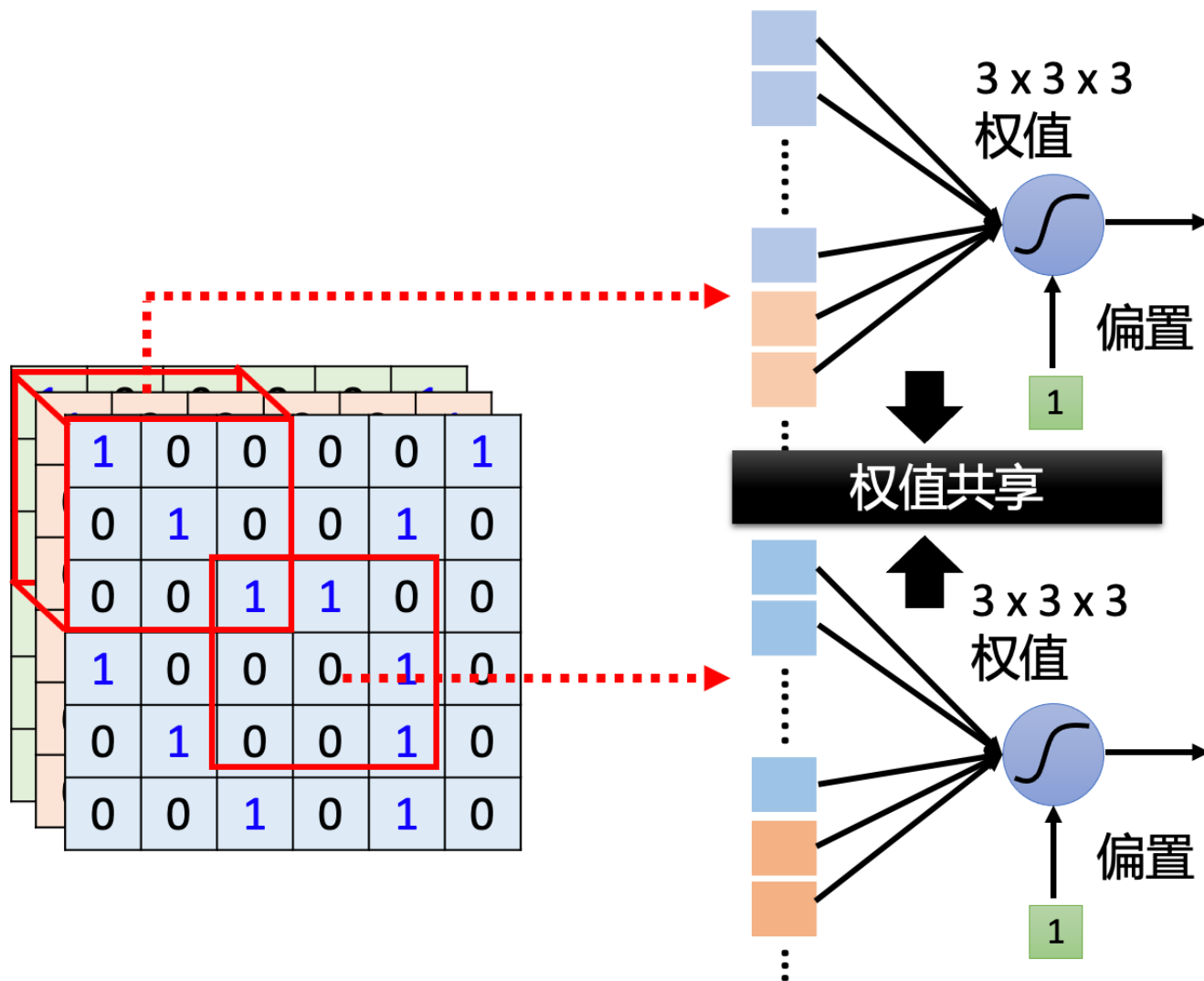


2. 卷积神经网络的工作原理



参数共享/权值共享

卷积层中使用参数共享是用来控制参数的数量。每个滤波器与上一层局部连接，同时每个滤波器的所有局部连接都使用同样的参数，减少网络的参数。



2. 卷积神经网络的工作原理

- 对图片进行采样不会改变图像中的物体



采样



➤ 池化 (Pooling)

0	1	2
3	4	5
6	7	8

2*2池化, stride: 1

Max Pooling
最大池化层

4	

Average Pooling
平均池化层

2	

2. 卷积神经网络的工作原理

- 对图片进行采样不会改变图像中的物体



采样



➤ 池化 (Pooling)

0	1	2
3	4	5
6	7	8

2*2池化, stride: 1

Max Pooling
最大池化层

4	5

Average Pooling
平均池化层

2	3

2. 卷积神经网络的工作原理

- 对图片进行采样不会改变图像中的物体



- **池化 (Pooling)** : 缩小图片尺寸, 减少最后连接层的参数量, 加快计算防止过拟合

0	1	2
3	4	5
6	7	8

2*2池化, stride: 1

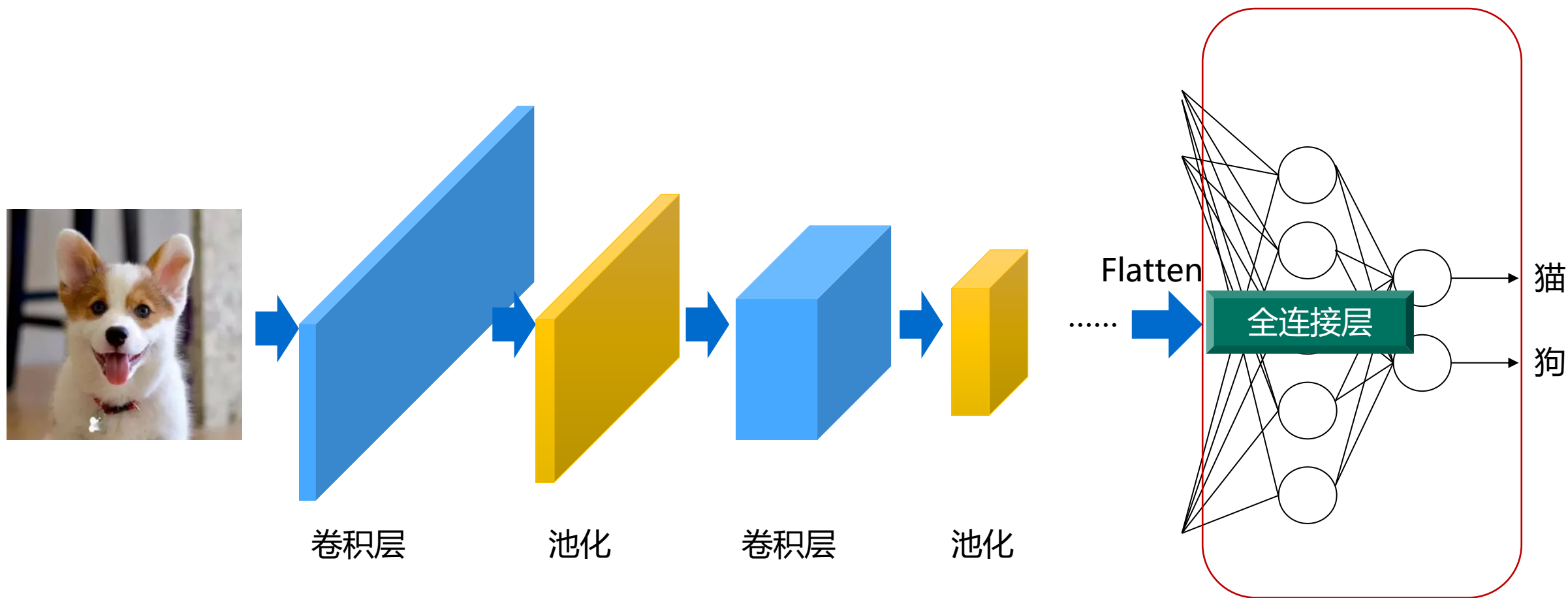
Max Pooling
最大池化层

4	5
7	8

Average Pooling
平均池化层

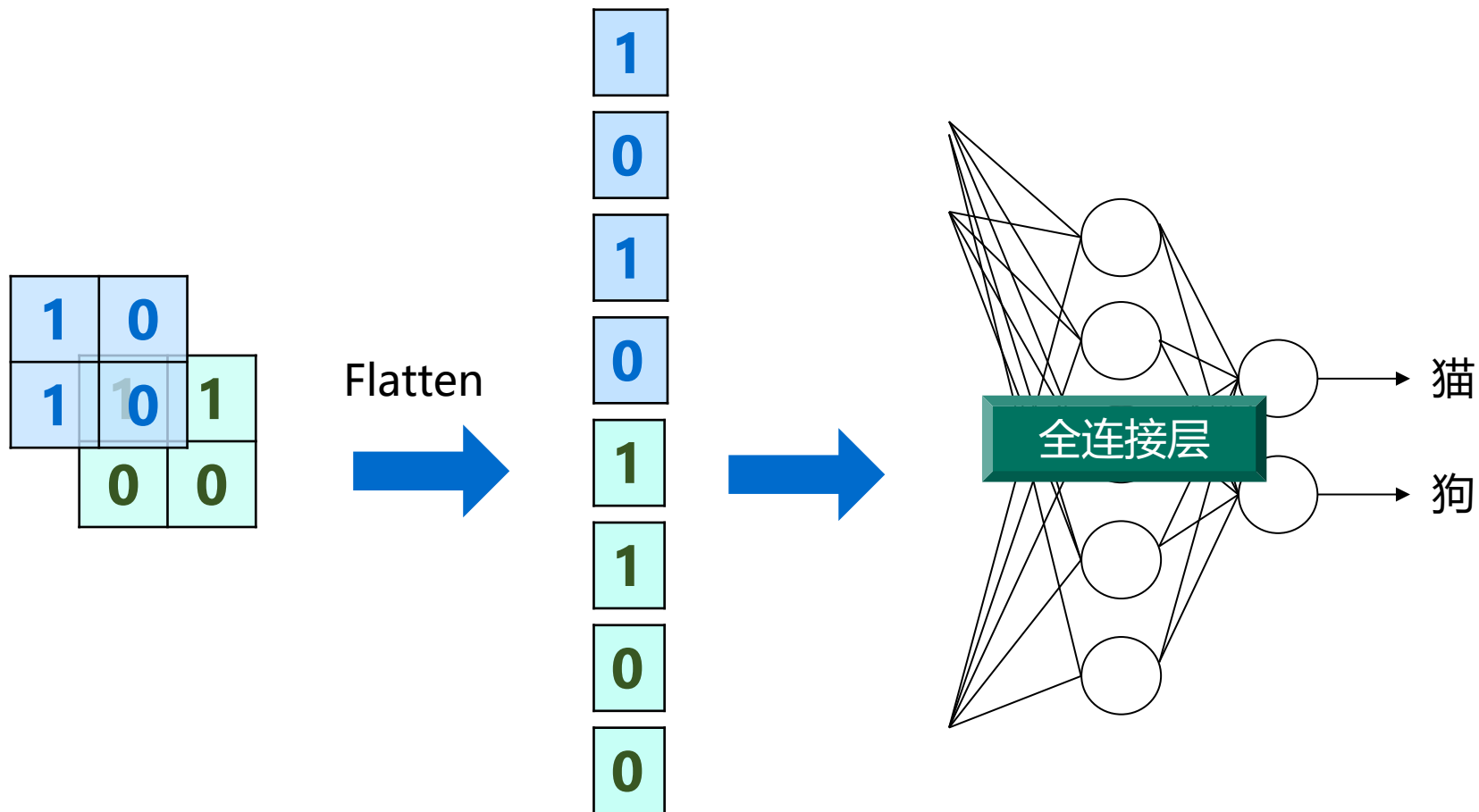
2	3
5	6

3.卷积神经网络的结构



3.卷积神经网络的结构

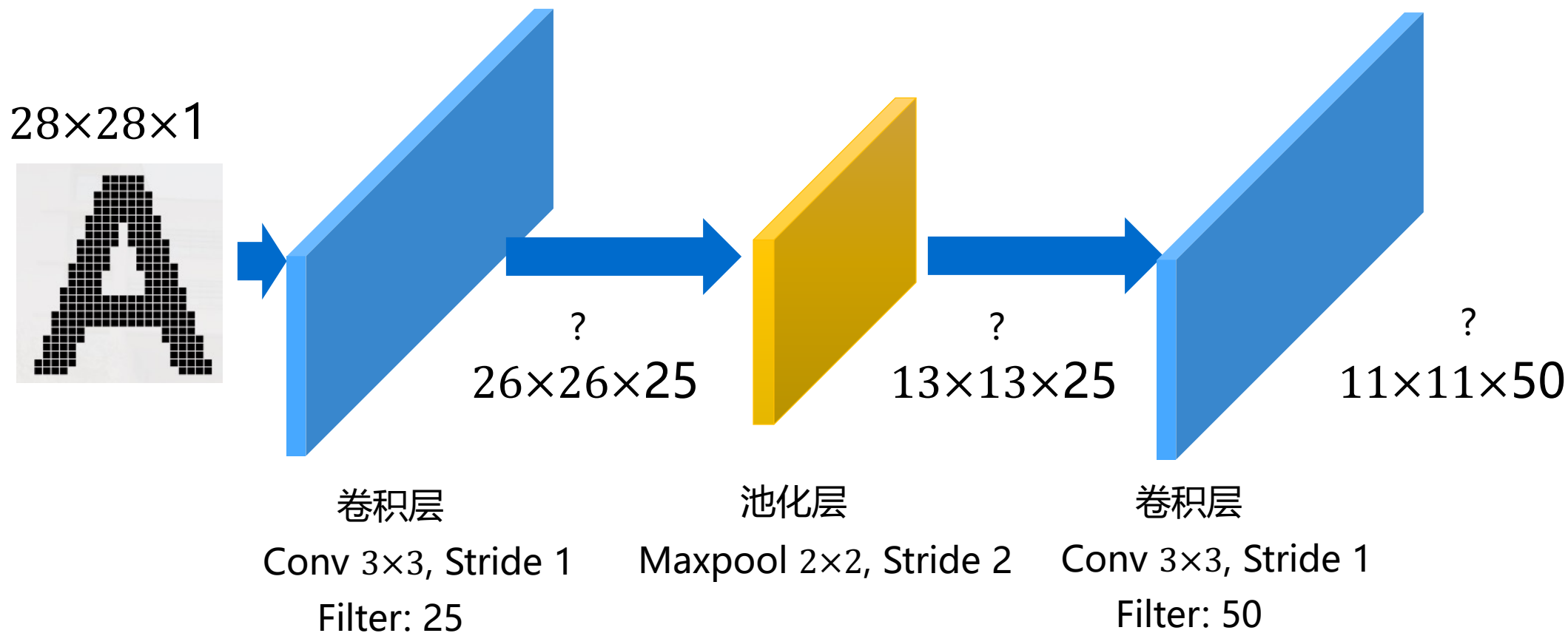
➤张量扁平化 (Flatten)



3.卷积神经网络的结构

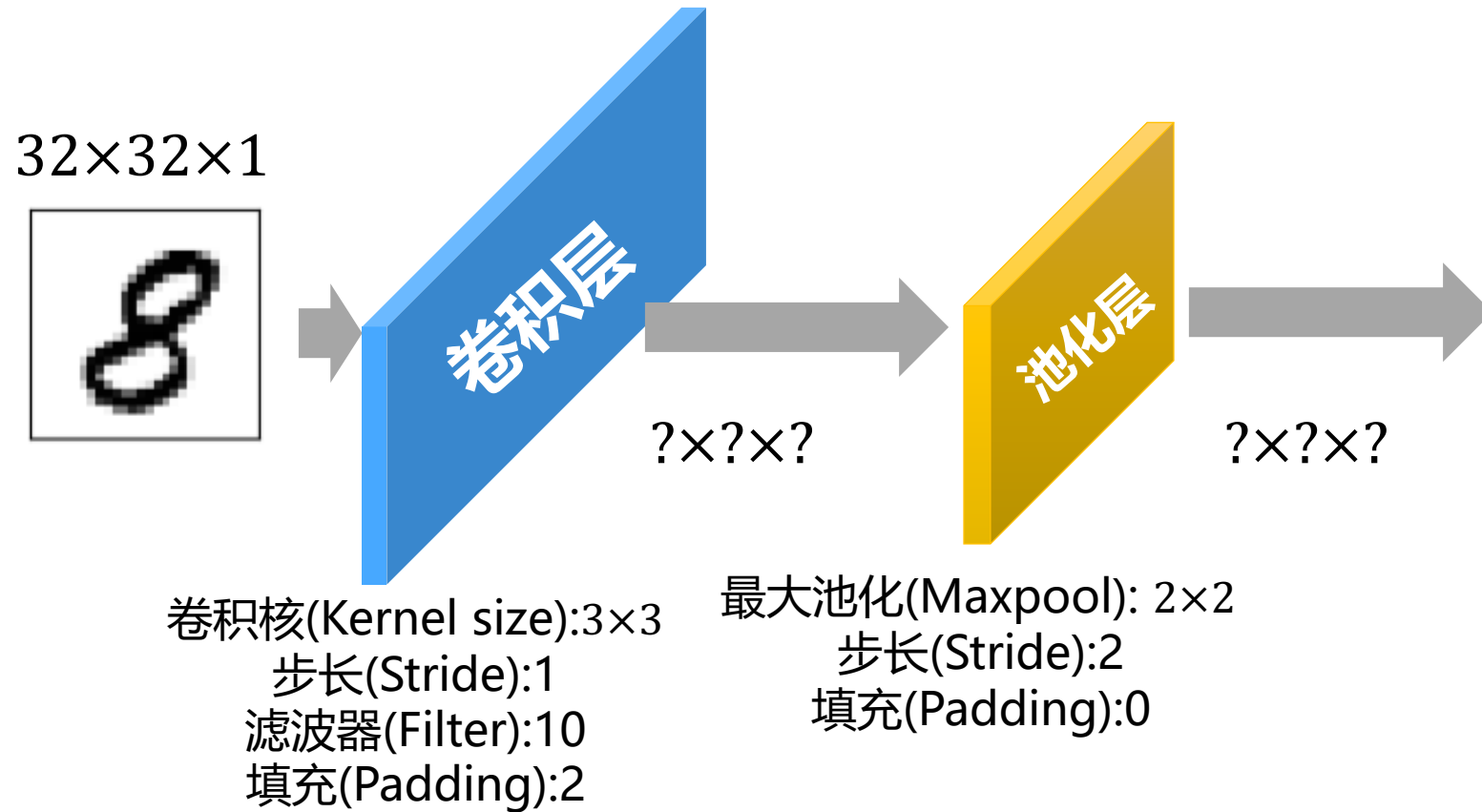


➤ 测试：数据维度和参数：每一层输出的数据的通道数×长×宽？

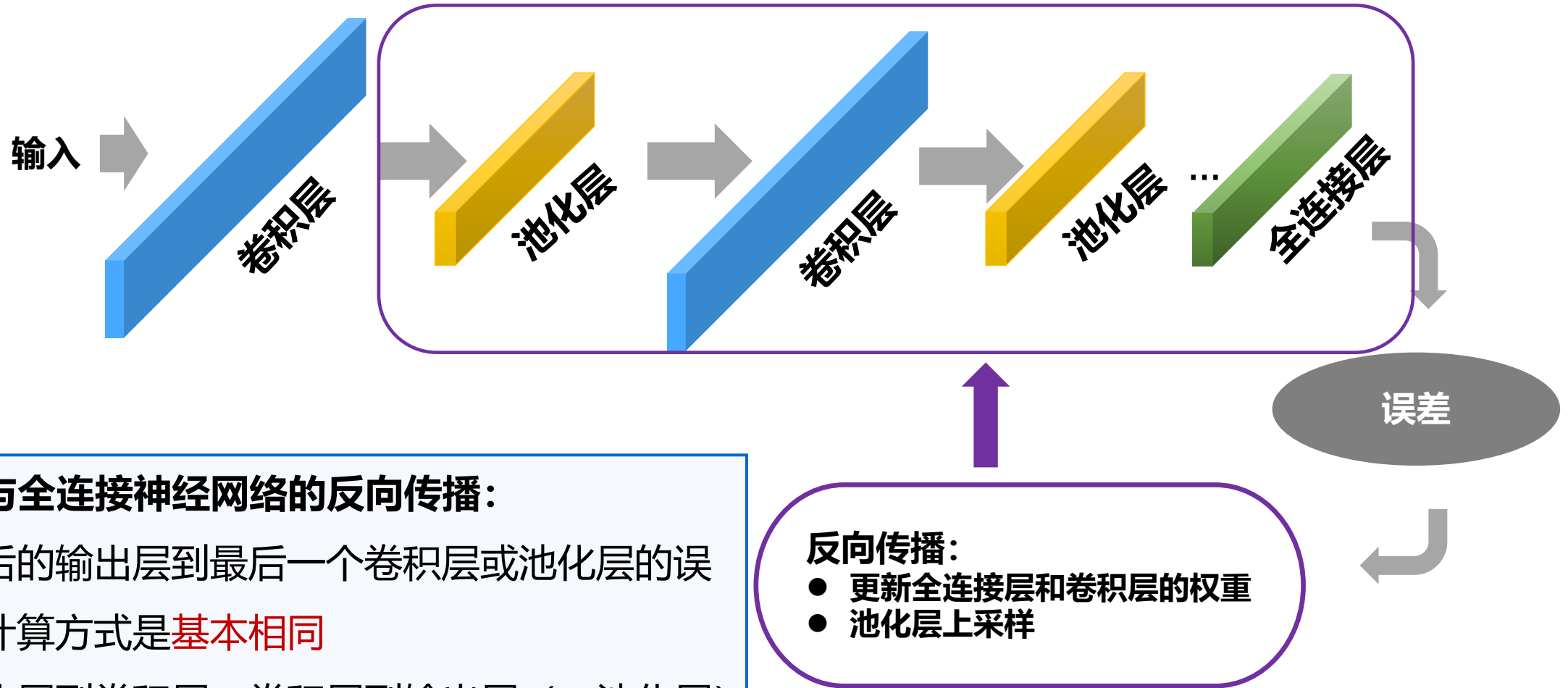


3. 卷积神经网络的结构

➤ 测试：数据维度和参数：每一层输出的数据的通道数×长×宽？



4. 卷积神经网络的反向传播



CNN与全连接神经网络的反向传播：

- 最后的输出层到最后一个卷积层或池化层的误差计算方式是**基本相同**
- 池化层到卷积层，卷积层到输出层（or池化层）的误差传播方式与全连接网络是**大不相同**

4.卷积神经网络的反向传播

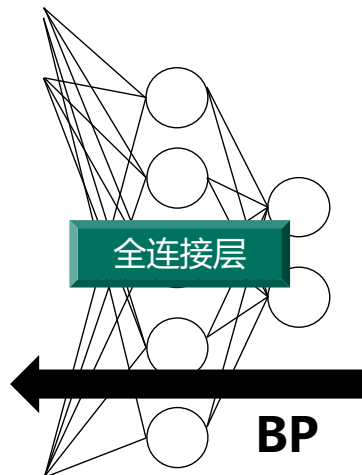
与全连接神经网络的反向传播
基本一致

损失函数为二分类交叉熵损失函数($N=2$): $J = -(y \log a^L + (1 - y) \log(1 - a^L))$

y 为真实标签, a^L 为 L 层激活函数softmax的输出 $a^L = \frac{e^{z_k^L}}{\sum_{j=1}^N e^{z_j^L}}$

则最高层误差: $\frac{\partial J}{\partial z^L} = a^L - y$

全连接层



输入: 训练集 $D = \{(x_k, y_k)\}_{k=1}^m$;
学习率 η .

过程:

- 1: 在(0, 1)范围内随机初始化网络中所有连接权和阈值
- 2: repeat
- 3: for all $(x_k, y_k) \in D$ do
- 4: 根据当前参数和式(5.3) 计算当前样本的输出 $\hat{y}_k$;
- 5: 根据式(5.10) 计算输出层神经元的梯度项 g_j ;
- 6: 根据式(5.15) 计算隐层神经元的梯度项 e_h ;
- 7: 根据式(5.11)-(5.14) 更新连接权 w_{hj} , v_{ih} 与阈值 θ_j , γ_h
- 8: end for
- 9: until 达到停止条件

输出: 连接权与阈值确定的多层前馈神经网络

图 5.8 误差逆传播算法

池化层

假设池化层为 l 层, 卷积层为 $l - 1$ 层, 从上层传到 l 层的梯度为 $\frac{\partial J}{\partial z^l}$

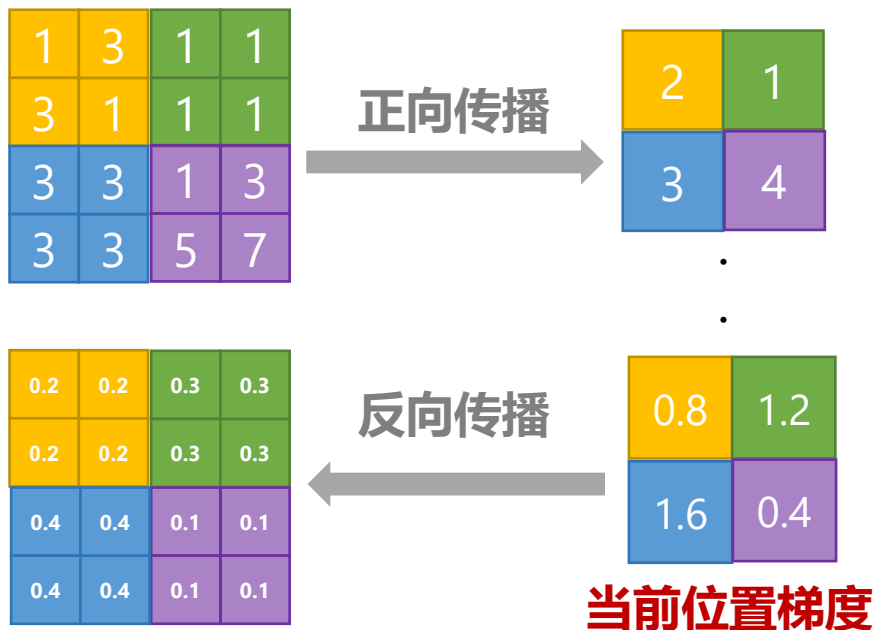
4. 卷积神经网络的反向传播

● **池化层** 假设池化层为 l 层，卷积层为 $l-1$ 层，从上层传到 l 层的梯度为 $\frac{\partial L}{\partial z^l}$ ，则：

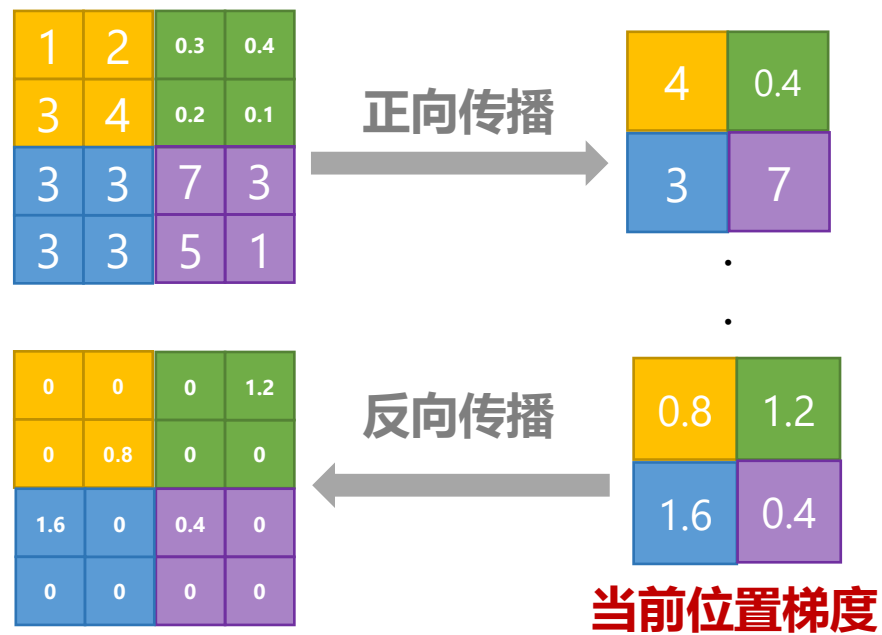
$$\frac{\partial J}{\partial z^{l-1}} = \frac{\partial J}{\partial z^l} \cdot \frac{\partial z^l}{\partial a^{l-1}} \cdot \frac{\partial a^{l-1}}{\partial z^{l-1}} = \boxed{\text{upsample}(\frac{\partial J}{\partial z^l})} \cdot \boxed{\frac{\partial a^{l-1}}{\partial z^{l-1}}} \quad \text{卷积层激活函数的偏导}$$

对当前位置梯度上采样

平均池化反向传播：
将当前位的梯度值均分



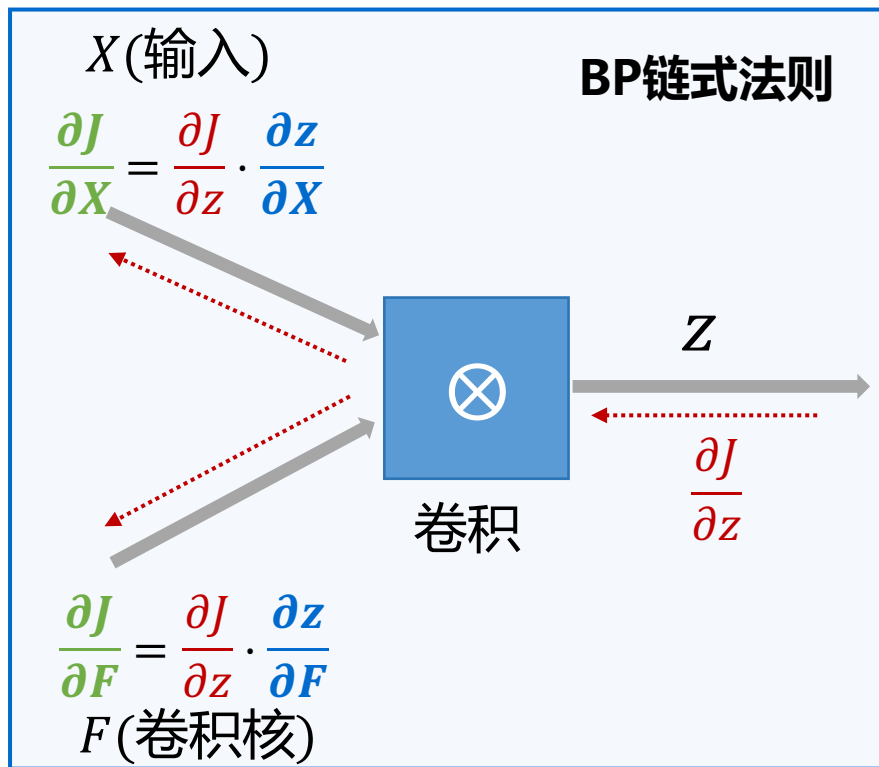
最大池化反向传播：
记录前向传播最大值位置索引，将当前位置梯度填入对应位置



其余元素的梯
度应置为0

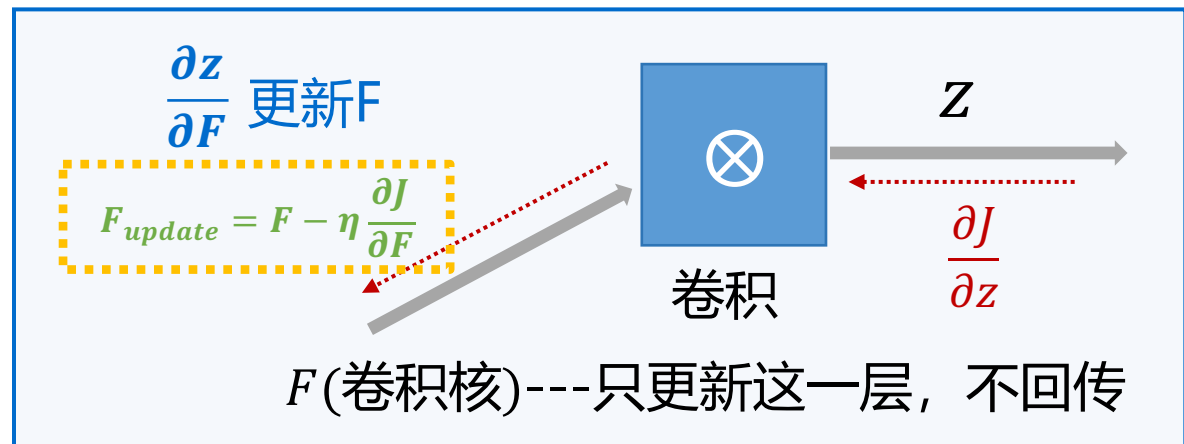
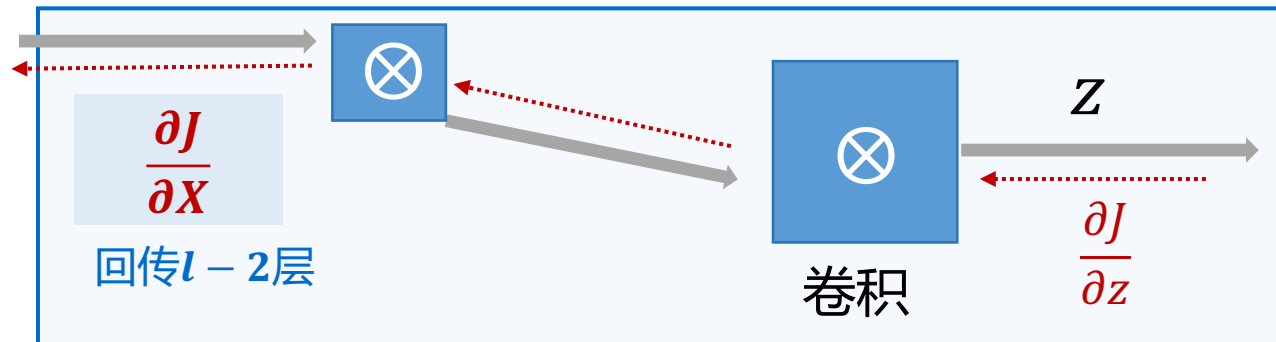
4.卷积神经网络的反向传播

- **卷积层** 卷积层为 $l-1$ 层, 上层经 l 层池化层回传至 $l-1$ 层的梯度为 $\frac{\partial J}{\partial z^{l-1}}$, 为方便表示后统一写为 $\frac{\partial J}{\partial z}$



需要求解

$$\frac{\partial z}{\partial X} \quad \frac{\partial z}{\partial F}$$



4. 卷积神经网络的反向传播

● **卷积层** 卷积层为 $l-1$ 层, 上层经 l 层池化层回传至 $l-1$ 层的梯度为 $\frac{\partial J}{\partial z^{l-1}}$, 为方便表示后统一写为 $\frac{\partial J}{\partial z}$

正向传播

$$\begin{bmatrix} z_{11} & z_{12} \\ z_{21} & z_{22} \end{bmatrix} = \text{卷积} \left(\begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix}, \begin{bmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{bmatrix} \right)$$

$X(\text{输入})$ $F(\text{卷积核})$

$$\begin{aligned}
 z_{11} &= X_{11}F_{11} + X_{12}F_{12} + X_{21}F_{21} + X_{22}F_{22} \\
 z_{12} &= X_{12}F_{11} + X_{13}F_{12} + X_{22}F_{21} + X_{23}F_{22} \\
 z_{21} &= X_{21}F_{11} + X_{22}F_{12} + X_{31}F_{21} + X_{32}F_{22} \\
 z_{22} &= X_{22}F_{11} + X_{23}F_{12} + X_{32}F_{21} + X_{33}F_{22}
 \end{aligned}$$

4.卷积神经网络的反向传播

● **卷积层** 卷积层为 $l-1$ 层, 上层经 l 层池化层回传至 $l-1$ 层的梯度为 $\frac{\partial J}{\partial z^{l-1}}$, 为方便表示后统一写为 $\frac{\partial J}{\partial z}$

求解 $\frac{\partial J}{\partial F} = \frac{\partial J}{\partial z} \cdot \frac{\partial z}{\partial F}$, 即求解 $\frac{\partial z}{\partial F}$

$$z_{11} = X_{11}F_{11} + X_{12}F_{12} + X_{21}F_{21} + X_{22}F_{22}$$

$$z_{12} = X_{12}F_{11} + X_{13}F_{12} + X_{22}F_{21} + X_{23}F_{22}$$

$$z_{21} = X_{21}F_{11} + X_{22}F_{12} + X_{31}F_{21} + X_{32}F_{22}$$

$$z_{22} = X_{22}F_{11} + X_{23}F_{12} + X_{32}F_{21} + X_{33}F_{22}$$

$$\begin{bmatrix} z_{11} & z_{12} \\ z_{21} & z_{22} \end{bmatrix} = \text{卷积} \left(\begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix}, \begin{bmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{bmatrix} \right)$$

$X(\text{输入}) \quad F(\text{卷积核})$

$$\frac{\partial J}{\partial F_i} = \sum_{k=1}^K \frac{\partial J}{\partial z_k} \cdot \frac{\partial z_k}{\partial F_i}$$

$$\frac{\partial J}{\partial F_{11}} = \frac{\partial J}{\partial z_{11}} \cdot \frac{\partial z_{11}}{\partial F_{11}} + \frac{\partial J}{\partial z_{12}} \cdot \frac{\partial z_{12}}{\partial F_{11}} + \frac{\partial J}{\partial z_{21}} \cdot \frac{\partial z_{21}}{\partial F_{11}} + \frac{\partial J}{\partial z_{22}} \cdot \frac{\partial z_{22}}{\partial F_{11}}$$

$$\frac{\partial J}{\partial F_{12}} = \frac{\partial J}{\partial z_{11}} \cdot \frac{\partial z_{11}}{\partial F_{12}} + \frac{\partial J}{\partial z_{12}} \cdot \frac{\partial z_{12}}{\partial F_{12}} + \frac{\partial J}{\partial z_{21}} \cdot \frac{\partial z_{21}}{\partial F_{12}} + \frac{\partial J}{\partial z_{22}} \cdot \frac{\partial z_{22}}{\partial F_{12}}$$

$$\frac{\partial J}{\partial F_{21}} = \frac{\partial J}{\partial z_{11}} \cdot \frac{\partial z_{11}}{\partial F_{21}} + \frac{\partial J}{\partial z_{12}} \cdot \frac{\partial z_{12}}{\partial F_{21}} + \frac{\partial J}{\partial z_{21}} \cdot \frac{\partial z_{21}}{\partial F_{21}} + \frac{\partial J}{\partial z_{22}} \cdot \frac{\partial z_{22}}{\partial F_{21}}$$

$$\frac{\partial J}{\partial F_{22}} = \frac{\partial J}{\partial z_{11}} \cdot \frac{\partial z_{11}}{\partial F_{22}} + \frac{\partial J}{\partial z_{12}} \cdot \frac{\partial z_{12}}{\partial F_{22}} + \frac{\partial J}{\partial z_{21}} \cdot \frac{\partial z_{21}}{\partial F_{22}} + \frac{\partial J}{\partial z_{22}} \cdot \frac{\partial z_{22}}{\partial F_{22}}$$

$$\frac{\partial J}{\partial F_i} = \sum_{k=1}^K \frac{\partial J}{\partial z_k} \cdot \frac{\partial z_k}{\partial F_i}$$

$$\frac{\partial J}{\partial F_{11}} = \frac{\partial J}{\partial z_{11}} \cdot X_{11} + \frac{\partial J}{\partial z_{12}} \cdot X_{12} + \frac{\partial J}{\partial z_{21}} \cdot X_{21} + \frac{\partial J}{\partial z_{22}} \cdot X_{22}$$

$$\frac{\partial J}{\partial F_{12}} = \frac{\partial J}{\partial z_{11}} \cdot X_{12} + \frac{\partial J}{\partial z_{12}} \cdot X_{13} + \frac{\partial J}{\partial z_{21}} \cdot X_{22} + \frac{\partial J}{\partial z_{22}} \cdot X_{23}$$

$$\frac{\partial J}{\partial F_{13}} = \frac{\partial J}{\partial z_{11}} \cdot X_{21} + \frac{\partial J}{\partial z_{12}} \cdot X_{22} + \frac{\partial J}{\partial z_{21}} \cdot X_{31} + \frac{\partial J}{\partial z_{22}} \cdot X_{32}$$

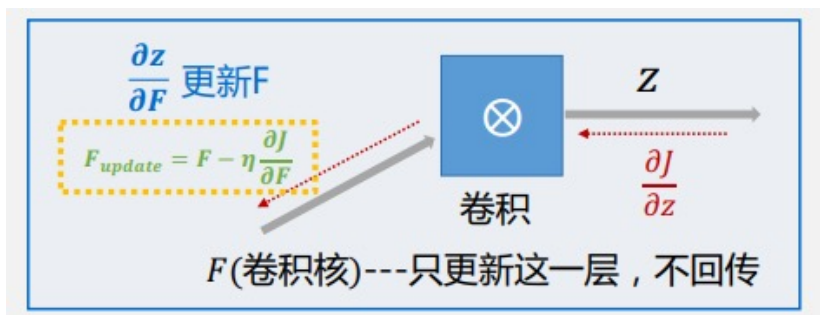
$$\frac{\partial J}{\partial F_{14}} = \frac{\partial J}{\partial z_{11}} \cdot X_{22} + \frac{\partial J}{\partial z_{12}} \cdot X_{23} + \frac{\partial J}{\partial z_{21}} \cdot X_{32} + \frac{\partial J}{\partial z_{22}} \cdot X_{33}$$

$X(\text{输入})$ 与 $\frac{\partial J}{\partial z}$ 的卷积

4.卷积神经网络的反向传播

● **卷积层** 卷积层为 $l - 1$ 层, 上层经 l 层池化层回传至 $l - 1$ 层的梯度为 $\frac{\partial J}{\partial z^{l-1}}$, 为方便表示后统一写为 $\frac{\partial J}{\partial z}$

求解 $\frac{\partial J}{\partial F} = \frac{\partial J}{\partial z} \cdot \frac{\partial z}{\partial F}$, 即求解 $\frac{\partial z}{\partial F}$



$$\begin{bmatrix} \frac{\partial J}{\partial F_{11}} & \frac{\partial J}{\partial F_{12}} \\ \frac{\partial J}{\partial F_{21}} & \frac{\partial J}{\partial F_{22}} \end{bmatrix} = \text{卷积}$$

$$\begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix}$$

$X(\text{输入})$

$$\begin{bmatrix} \frac{\partial J}{\partial z_{11}} & \frac{\partial J}{\partial z_{12}} \\ \frac{\partial J}{\partial z_{21}} & \frac{\partial J}{\partial z_{22}} \end{bmatrix}$$

回传的梯度

$$\frac{\partial J}{\partial F_i} = \sum_{k=1}^K \frac{\partial J}{\partial z_k} \cdot \frac{\partial z_k}{\partial F_i}$$

$$\begin{cases} \frac{\partial J}{\partial F_{11}} = \frac{\partial J}{\partial z_{11}} \cdot X_{11} + \frac{\partial J}{\partial z_{12}} \cdot X_{12} + \frac{\partial J}{\partial z_{21}} \cdot X_{21} + \frac{\partial J}{\partial z_{22}} \cdot X_{22} \\ \frac{\partial J}{\partial F_{12}} = \frac{\partial J}{\partial z_{11}} \cdot X_{12} + \frac{\partial J}{\partial z_{12}} \cdot X_{13} + \frac{\partial J}{\partial z_{21}} \cdot X_{22} + \frac{\partial J}{\partial z_{22}} \cdot X_{23} \\ \frac{\partial J}{\partial F_{13}} = \frac{\partial J}{\partial z_{11}} \cdot X_{21} + \frac{\partial J}{\partial z_{12}} \cdot X_{22} + \frac{\partial J}{\partial z_{21}} \cdot X_{31} + \frac{\partial J}{\partial z_{22}} \cdot X_{32} \\ \frac{\partial J}{\partial F_{14}} = \frac{\partial J}{\partial z_{11}} \cdot X_{22} + \frac{\partial J}{\partial z_{12}} \cdot X_{23} + \frac{\partial J}{\partial z_{21}} \cdot X_{32} + \frac{\partial J}{\partial z_{22}} \cdot X_{33} \end{cases}$$

$X(\text{输入})$ 与 $\frac{\partial J}{\partial z}$ 的卷积

举例

$$\begin{bmatrix} 0.3 & 1.4 \\ 1.2 & -1.2 \end{bmatrix} = \text{卷积}$$

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 3 & 4 \\ 1 & 1 & 1 \end{bmatrix}$$

$X(\text{输入})$

$$\begin{bmatrix} 0.1 & -0.5 \\ 0.3 & 0.2 \end{bmatrix}$$

回传的梯度

4.卷积神经网络的反向传播

● **卷积层** 卷积层为 $l-1$ 层, 上层经 l 层池化层回传至 $l-1$ 层的梯度为 $\frac{\partial J}{\partial z^{l-1}}$, 为方便表示后统一写为 $\frac{\partial J}{\partial z}$

求解 $\frac{\partial J}{\partial X} = \frac{\partial J}{\partial z} \cdot \frac{\partial z}{\partial X}$, 即求解 $\frac{\partial z}{\partial X}$

$$z_{11} = X_{11}F_{11} + X_{12}F_{12} + X_{21}F_{21} + X_{22}F_{22}$$

$$z_{12} = X_{12}F_{11} + X_{13}F_{12} + X_{22}F_{21} + X_{23}F_{22}$$

$$z_{21} = X_{21}F_{11} + X_{22}F_{12} + X_{31}F_{21} + X_{32}F_{22}$$

$$z_{22} = X_{22}F_{11} + X_{23}F_{12} + X_{32}F_{21} + X_{33}F_{22}$$

$$\begin{bmatrix} z_{11} & z_{12} \\ z_{21} & z_{22} \end{bmatrix} = \text{卷积} \left(\begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix}, \begin{bmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{bmatrix} \right)$$

X (输入) F (卷积核)

$$\frac{\partial J}{\partial X_i} = \sum_{k=1}^K \frac{\partial J}{\partial z_k} \cdot \frac{\partial z_k}{\partial X_i}$$

$$\frac{\partial J}{\partial X_{11}} = \frac{\partial J}{\partial z_{11}} \cdot F_{11}$$

$$\frac{\partial J}{\partial X_{12}} = \frac{\partial J}{\partial z_{11}} \cdot F_{12} + \frac{\partial J}{\partial z_{12}} \cdot F_{11}$$

$$\frac{\partial J}{\partial X_{13}} = \frac{\partial J}{\partial z_{12}} \cdot F_{12}$$

$$\frac{\partial J}{\partial X_{21}} = \frac{\partial J}{\partial z_{11}} \cdot F_{21} + \frac{\partial J}{\partial z_{21}} \cdot F_{11}$$

$$\frac{\partial J}{\partial X_{22}} = \frac{\partial J}{\partial z_{11}} \cdot F_{22} + \frac{\partial J}{\partial z_{12}} \cdot F_{21} + \frac{\partial J}{\partial z_{21}} \cdot F_{12} + \frac{\partial J}{\partial z_{22}} \cdot F_{11}$$

$$\frac{\partial J}{\partial X_{23}} = \frac{\partial J}{\partial z_{12}} \cdot F_{22} + \frac{\partial J}{\partial z_{22}} \cdot F_{12}$$

$$\frac{\partial J}{\partial X_{31}} = \frac{\partial J}{\partial z_{21}} \cdot F_{21}$$

$$\frac{\partial J}{\partial X_{32}} = \frac{\partial J}{\partial z_{21}} \cdot F_{22} + \frac{\partial J}{\partial z_{22}} \cdot F_{21}$$

$$\frac{\partial J}{\partial X_{33}} = \frac{\partial J}{\partial z_{21}} \cdot F_{22}$$

4.卷积神经网络的反向传播

● **卷积层** 卷积层为 $l-1$ 层, 上层经 l 层池化层回传至 $l-1$ 层的梯度为 $\frac{\partial J}{\partial z^{l-1}}$, 为方便表示后统一写为 $\frac{\partial J}{\partial z}$

以 x 表达 z^{l-2} , 则求解 $\frac{\partial J}{\partial x} = \frac{\partial J}{\partial z} \cdot \frac{\partial z}{\partial x}$, 即求解 $\frac{\partial z}{\partial x}$

$$\begin{aligned}\frac{\partial J}{\partial x_{11}} &= \frac{\partial J}{\partial z_{11}} \cdot F_{11} \\ \frac{\partial J}{\partial x_{12}} &= \frac{\partial J}{\partial z_{11}} \cdot F_{12} + \frac{\partial J}{\partial z_{12}} \cdot F_{11} \\ \frac{\partial J}{\partial x_{13}} &= \frac{\partial J}{\partial z_{12}} \cdot F_{12} \\ \frac{\partial J}{\partial x_{21}} &= \frac{\partial J}{\partial z_{11}} \cdot F_{21} + \frac{\partial J}{\partial z_{21}} \cdot F_{11} \\ \frac{\partial J}{\partial x_{22}} &= \frac{\partial J}{\partial z_{11}} \cdot F_{22} + \frac{\partial J}{\partial z_{12}} \cdot F_{21} + \frac{\partial J}{\partial z_{21}} \cdot F_{12} + \frac{\partial J}{\partial z_{22}} \cdot F_{11} \\ \frac{\partial J}{\partial x_{23}} &= \frac{\partial J}{\partial z_{12}} \cdot F_{22} + \frac{\partial J}{\partial z_{22}} \cdot F_{12} \\ \frac{\partial J}{\partial x_{31}} &= \frac{\partial J}{\partial z_{21}} \cdot F_{21} \\ \frac{\partial J}{\partial x_{32}} &= \frac{\partial J}{\partial z_{21}} \cdot F_{22} + \frac{\partial J}{\partial z_{22}} \cdot F_{21} \\ \frac{\partial J}{\partial x_{33}} &= \frac{\partial J}{\partial z_{22}} \cdot F_{22}\end{aligned}$$

F_{11}	F_{12}
F_{21}	F_{22}

旋转180°

F_{22}	F_{21}
F_{12}	F_{11}

$\frac{\partial J}{\partial x_{11}}$	$\frac{\partial J}{\partial x_{12}}$	$\frac{\partial J}{\partial x_{13}}$
$\frac{\partial J}{\partial x_{21}}$	$\frac{\partial J}{\partial x_{22}}$	$\frac{\partial J}{\partial x_{23}}$
$\frac{\partial J}{\partial x_{31}}$	$\frac{\partial J}{\partial x_{32}}$	$\frac{\partial J}{\partial x_{33}}$

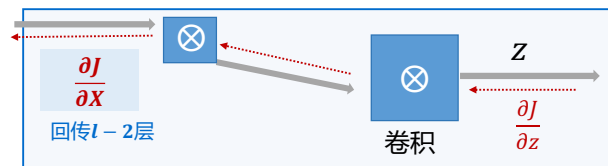
= 卷积

0	0	0	0
0	$\frac{\partial J}{\partial z_{11}}$	$\frac{\partial J}{\partial z_{12}}$	0
0	$\frac{\partial J}{\partial z_{21}}$	$\frac{\partial J}{\partial z_{22}}$	0
0	0	0	0

回传的梯度

F_{22}	F_{21}
F_{12}	F_{11}

F (卷积核)
旋转180°



举例

0.2	0.7	0.6
0.1	0.5	1.4
-1.5	-1.7	0.4

= 卷积

0	0	0	0
0	1	2	0
0	3	4	0
0	0	0	0

回传的梯度

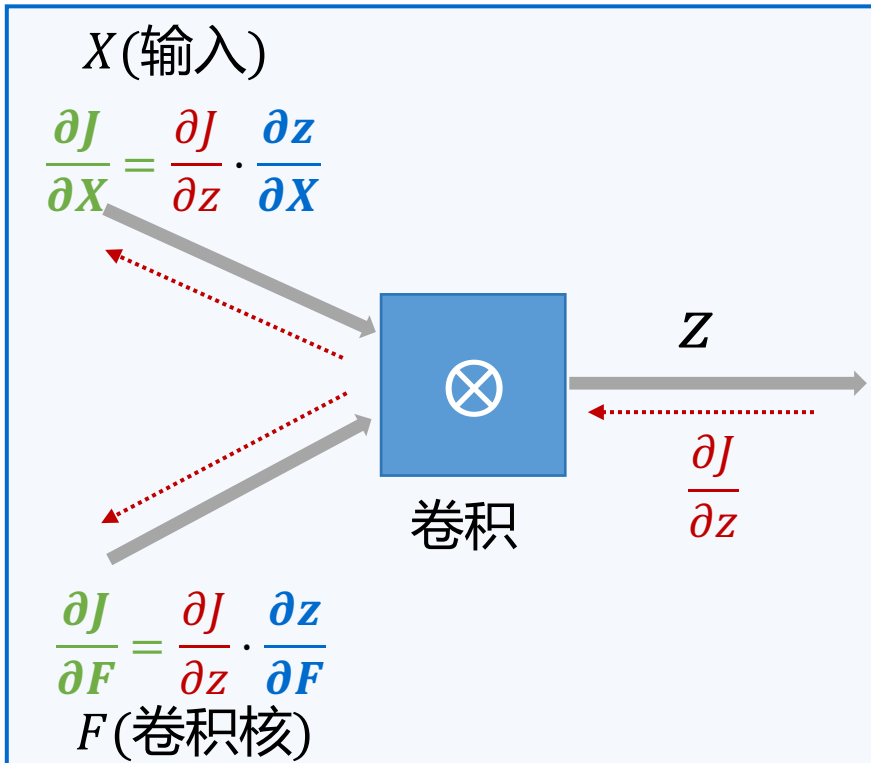
0.1	-0.5
0.3	0.2

F (卷积核)
旋转180°

4. 卷积神经网络的反向传播

● **卷积层** 卷积层为 $l-1$ 层，上层经 l 层池化层回传至 $l-1$ 层的梯度为 $\frac{\partial J}{\partial z^{l-1}}$ ，为方便表示后统一写为 $\frac{\partial J}{\partial z}$

总结卷积层反向传播：



$$\begin{aligned} \frac{\partial z}{\partial F} &= \text{卷积} \left[\begin{array}{l} X(\text{输入}), \text{回传的梯度} \end{array} \right] \\ \frac{\partial z}{\partial X} &= \text{卷积} \left[\begin{array}{l} \text{回传的梯度}, F(\text{卷积核}) \text{ 旋转 } 180^\circ \end{array} \right] \end{aligned}$$

5.卷积神经网络的模型训练

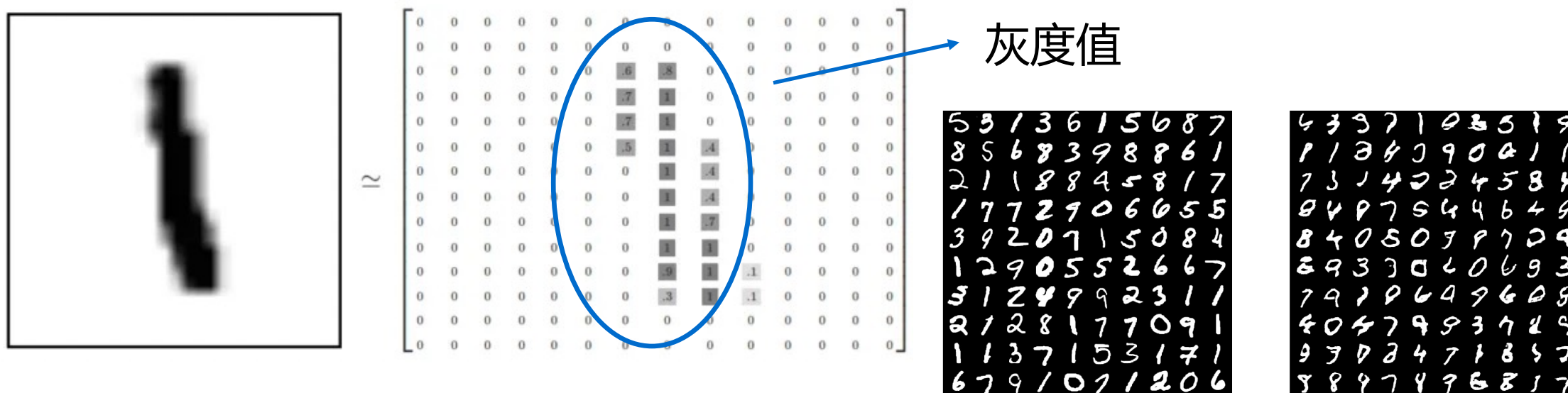
手写体数字识别任务：MNIST数据集 28×28 图像

- 全连接神经网络图像展平为一个784维一维向量

缺点： 维度大、参数多、过拟合

- 卷积神经网络：输入为 28×28 的单通道（黑白）图像，保留图像空间结构

优点： 模型更简洁、所需的参数更少



5.卷积神经网络的模型训练



手写体数字识别任务：MNIST数据集 28×28 图像

导入Package, torch、损失函数、优化器、数据集等

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
torch.__version__
```

定义超参数

```
BATCH_SIZE=512 #大概需要2G的显存
EPOCHS=20 # 总共训练批次
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# 让torch判断是否使用GPU, 建议使用GPU环境, 因为会快很多
```

5.卷积神经网络的模型训练



手写体数字识别任务：MNIST数据集 28×28图像

Pytorch里面包含了MNIST的数据集，可直接使用即可，使用DataLoader来对数据进行读取，数据进行标准化处理（transforms.Normalize）

完成训练集导入

```
train_loader = torch.utils.data.DataLoader(
    datasets.MNIST('data', train=True, download=True,
                   transform=transforms.Compose([
                       transforms.ToTensor(),
                       transforms.Normalize((0.1307,), (0.3081,))
                   ])),
    batch_size=BATCH_SIZE, shuffle=True)
```

完成测试集导入

```
test_loader = torch.utils.data.DataLoader(
    datasets.MNIST('data', train=False, transform=transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.1307,), (0.3081,))
    ])),
    batch_size=BATCH_SIZE, shuffle=True)
```

```
Downloading http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz
Downloading http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz
Downloading http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz
Downloading http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz
Processing...
Done!
```

5.卷积神经网络的模型训练



手写体数字识别任务：MNIST数据集 28×28图像

定义一个网络，网络包含两个卷积层，conv1和conv2，着两个线性层作为输出，最后输出10个维度，这10个维度我们作为0-9的标识来确定识别出的是哪个数字

```
class ConvNet(nn.Module):
    def __init__(self):
        super().__init__()
        # batch*1*28*28 (每次会送入batch个样本，输入通道数1 (黑白图像)，图像分辨率是28x28)
        # 下面的卷积层Conv2d的第一个参数指输入通道数，第二个参数指输出通道数，第三个参数指卷积核的大小
        self.conv1 = nn.Conv2d(1, 10, 5) # 输入通道数1，输出通道数10，核的大小5
        self.conv2 = nn.Conv2d(10, 20, 3) # 输入通道数10，输出通道数20，核的大小3
        # 下面的全连接层Linear的第一个参数指输入通道数，第二个参数指输出通道数
        self.fc1 = nn.Linear(20*10*10, 500) # 输入通道数是2000，输出通道数是500
        self.fc2 = nn.Linear(500, 10) # 输入通道数是500，输出通道数是10，即10分类
    def forward(self,x):
        in_size = x.size(0) # 在本例中in_size=512，也就是BATCH_SIZE的值。输入的x可以看成是512*1*28*28的张量。
        out = self.conv1(x) # batch*1*28*28 -> batch*10*24*24 (28x28的图像经过一次核为5x5的卷积，输出变为24x24)
        out = F.relu(out) # batch*10*24*24 (激活函数ReLU不改变形状)
        out = F.max_pool2d(out, 2, 2) # batch*10*24*24 -> batch*10*12*12 (2*2的池化层会减半)
        out = self.conv2(out) # batch*10*12*12 -> batch*20*10*10 (再卷积一次，核的大小是3)
        out = F.relu(out) # batch*20*10*10
        out = out.view(in_size, -1) # batch*20*10*10 -> batch*2000 (out的第二维是-1，说明是自动推算，本例中第二维是20*10*10)
        out = self.fc1(out) # batch*2000 -> batch*500
        out = F.relu(out) # batch*500
        out = self.fc2(out) # batch*500 -> batch*10
        out = F.log_softmax(out, dim=1) # 计算log(softmax(x))
        return out
```

计算Softmax输出，选择
概率最大的输出

Linear Output	Softmax $\frac{\exp(x_i)}{\sum_j \exp(x_j)}$
0.0	0.07
1.2	0.23
0.1	0.08
-0.3	0.05
0.2	0.09
0.1	0.08
0.1	0.08
0.9	0.17
0.1	0.08
-0.2	0.05

5.卷积神经网络的模型训练



手写体数字识别任务：MNIST数据集 28×28图像

优化器我们也直接选择Adam

```
model = ConvNet().to(DEVICE)
optimizer = optim.Adam(model.parameters())
```

定义训练函数，将训练的内容都封装到函数中

```
def train(model, device, train_loader, optimizer, epoch):
    model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = F.nll_loss(output, target)
        loss.backward()
        optimizer.step()
        if (batch_idx+1)%30 == 0:
            print('Train Epoch: {} [{}/{} ({:.0f}%)]\tLoss: {:.6f}'.format(
                epoch, batch_idx * len(data), len(train_loader.dataset),
                100. * batch_idx / len(train_loader), loss.item()))
```

损失函数

$$l(\theta) = - \sum_{i=1}^n (y_i \log \hat{y}_{\theta,i} + (1 - y_i) \log(1 - \hat{y}_{\theta,i}))$$

输出当前的训练损失loss

5.卷积神经网络的模型训练



手写体数字识别任务：MNIST数据集 28×28 图像

测试也一样封装成一个函数

```
def test(model, device, test_loader):  
    model.eval()  
    test_loss = 0  
    correct = 0  
    with torch.no_grad():  
        for data, target in test_loader:  
            data, target = data.to(device), target.to(device)  
            output = model(data)  
            test_loss += F.nll_loss(output, target, reduction='sum').item() # 将一批的损失相加  
            pred = output.max(1, keepdim=True)[1] # 找到概率最大的下标  
            correct += pred.eq(target.view_as(pred)).sum().item()  
  
    test_loss /= len(test_loader.dataset)  
    print('\nTest set: Average loss: {:.4f}, Accuracy: {}/{} ({:.0f}%) \n'.format(  
        test_loss, correct, len(test_loader.dataset),  
        100. * correct / len(test_loader.dataset)))
```

找到输出概率最大的标签，并计算
预测准确的标签个数

打印准确率

5.卷积神经网络的模型训练



手写体数字识别任务：MNIST数据集 28×28图像

输出训练和测试结果

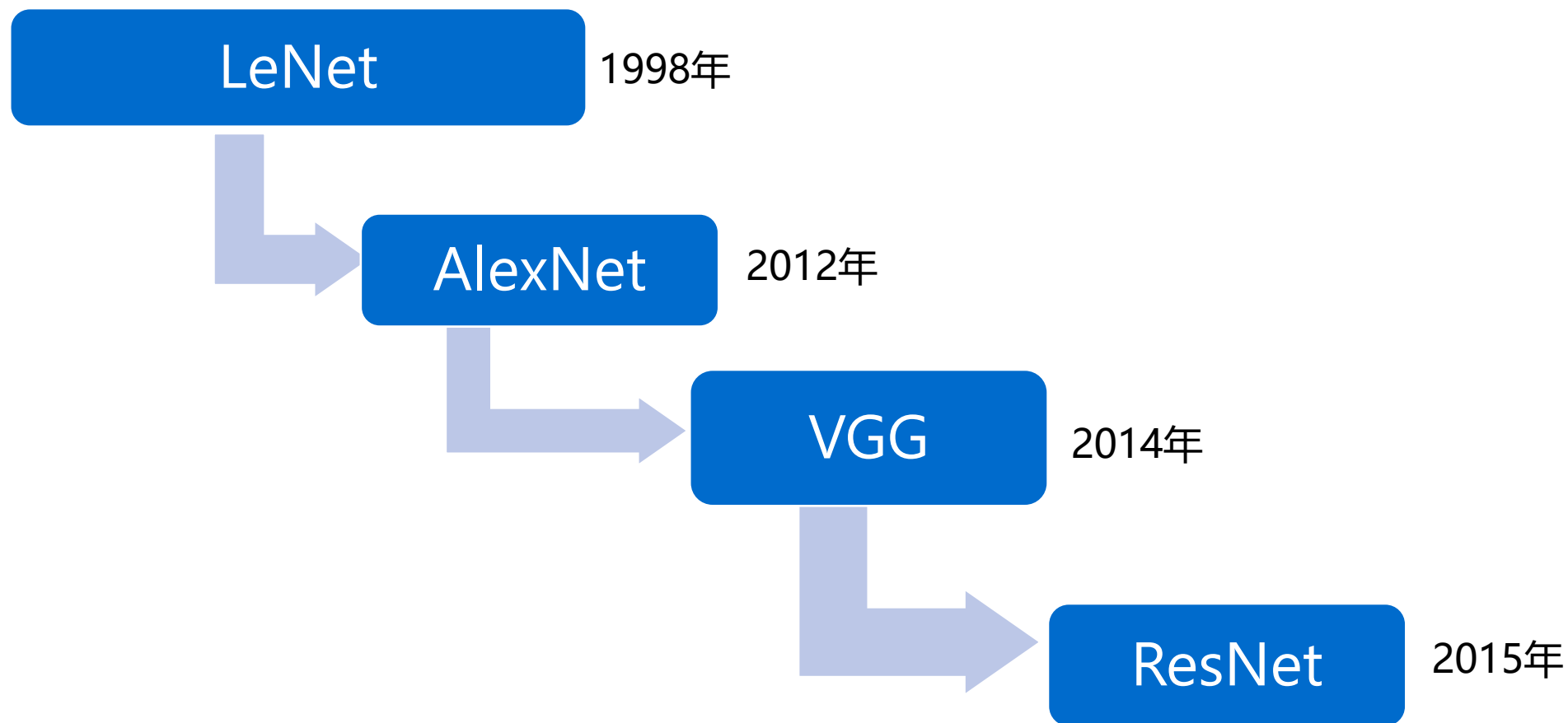
```
for epoch in range(1, EPOCHS + 1):  
    train(model, DEVICE, train_loader, optimizer, epoch)  
    test(model, DEVICE, test_loader)
```

打印结果如下：

```
Train Epoch: 1 [14848/60000 (25%)] Loss: 0.272529  
Train Epoch: 1 [30208/60000 (50%)] Loss: 0.235455  
Train Epoch: 1 [45568/60000 (75%)] Loss: 0.101858  
  
Test set: Average loss: 0.1018, Accuracy: 9695/10000 (97%)  
  
Train Epoch: 2 [14848/60000 (25%)] Loss: 0.057989  
Train Epoch: 2 [30208/60000 (50%)] Loss: 0.083935  
Train Epoch: 2 [45568/60000 (75%)] Loss: 0.051921  
  
Test set: Average loss: 0.0523, Accuracy: 9825/10000 (98%)  
  
Train Epoch: 3 [14848/60000 (25%)] Loss: 0.045383  
Train Epoch: 3 [30208/60000 (50%)] Loss: 0.049402  
Train Epoch: 3 [45568/60000 (75%)] Loss: 0.061366  
  
Test set: Average loss: 0.0408, Accuracy: 9866/10000 (99%)  
  
Train Epoch: 4 [14848/60000 (25%)] Loss: 0.035253  
Train Epoch: 4 [30208/60000 (50%)] Loss: 0.038444  
Train Epoch: 4 [45568/60000 (75%)] Loss: 0.036877  
  
Test set: Average loss: 0.0433, Accuracy: 9859/10000 (99%)  
  
Train Epoch: 5 [14848/60000 (25%)] Loss: 0.038996  
Train Epoch: 5 [30208/60000 (50%)] Loss: 0.020670  
Train Epoch: 5 [45568/60000 (75%)] Loss: 0.034658  
  
Test set: Average loss: 0.0339, Accuracy: 9885/10000 (99%)  
  
****
```

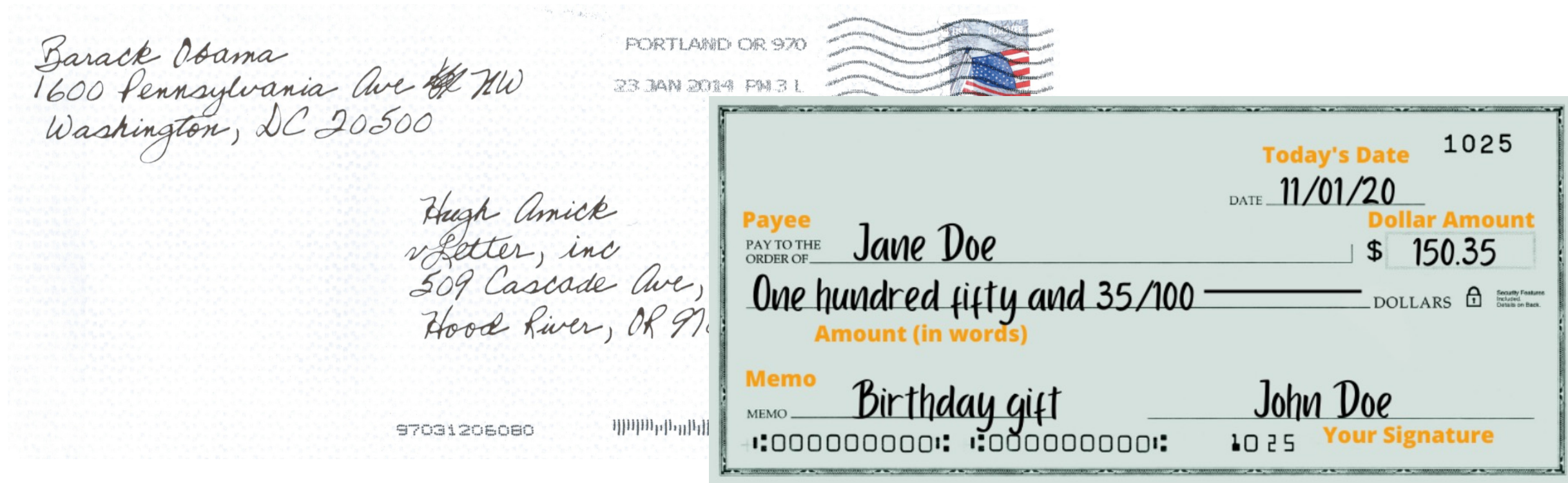
可以看到五个epoch后，测试集准确率为99%

6.卷积神经网络的经典结构



6.卷积神经网络的经典结构-LeNet

- LeNet: 在80年代末期在银行和邮局等识别手写数字处于支配的地位，名字来源于论文作者Y. LeCun



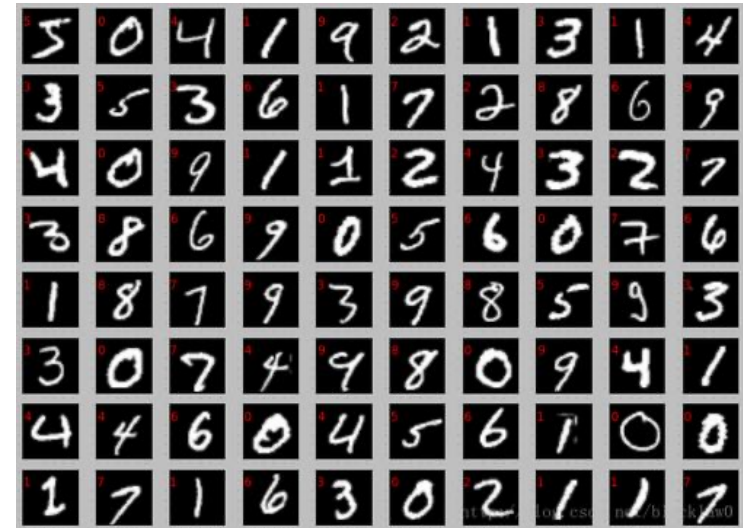
LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.

6.卷积神经网络的经典结构-LeNet

- LeNet: 在80年代末期在银行和邮局等**识别手写数字**处于支配的地位，名字来源于论文作者Y. LeCun
- LeNet展示了通过梯度下降训练卷积神经网络可以达到手写数字识别在当时最好的结果
- 是卷积神经网络奠基性的工作，CNN的三个特性：1.局部感知、2.下采样、3.权值共享

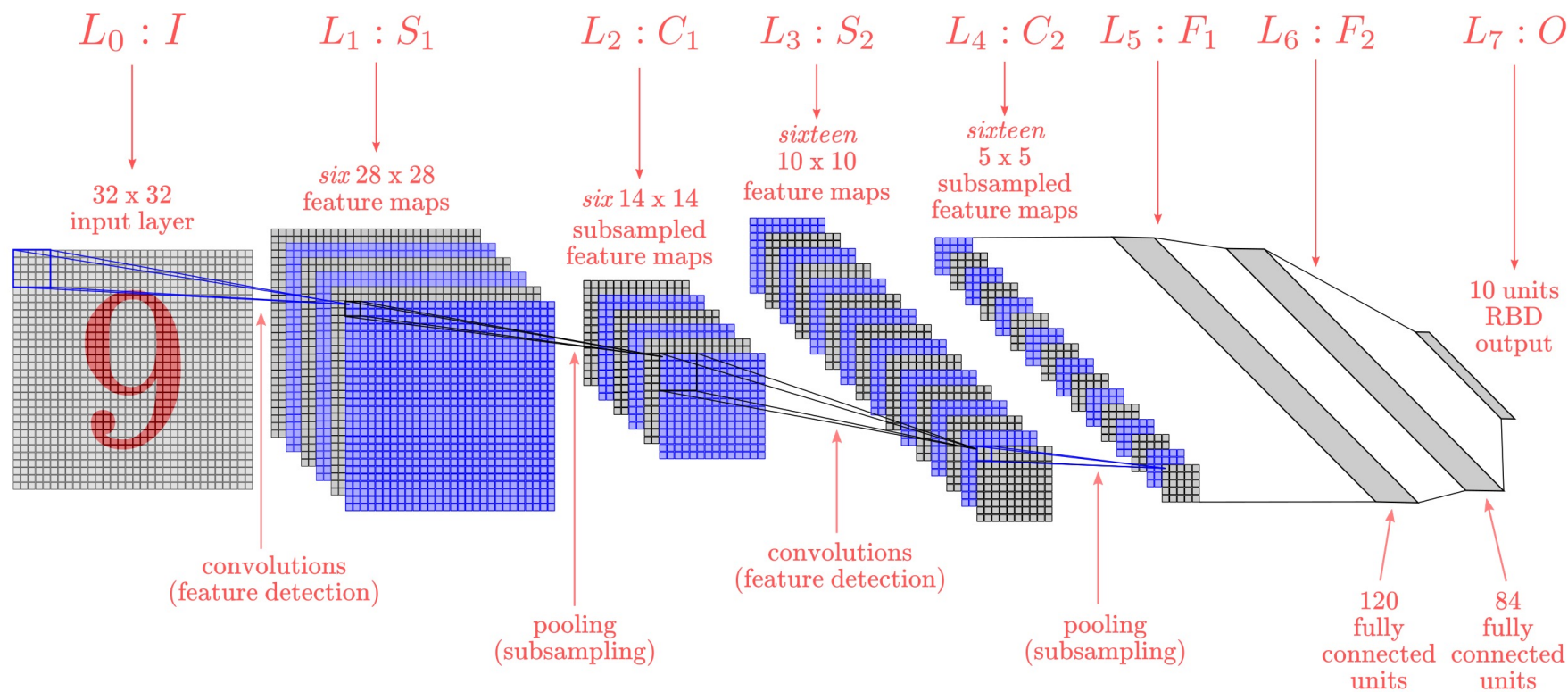
MNIST数据集

手写体数字的图片数据集：<http://yann.lecun.com/exdb/mnist/>
 训练集一共包含了 60,000 (28*28) 张图像和标签
 测试集一共包含了 10,000 (28*28) 张图像和标签
 10分类问题



LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.

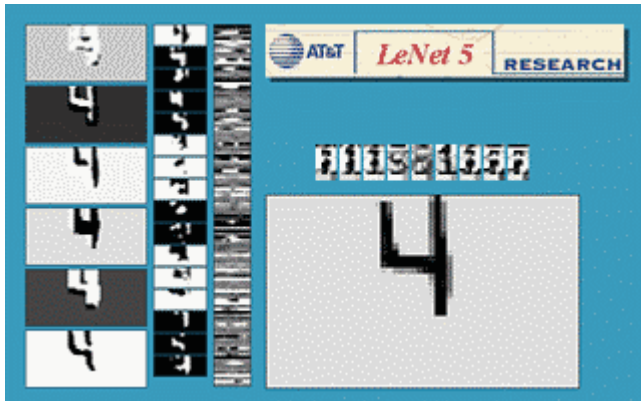
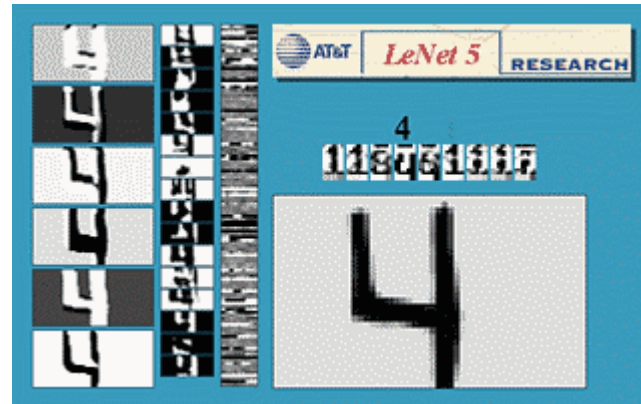
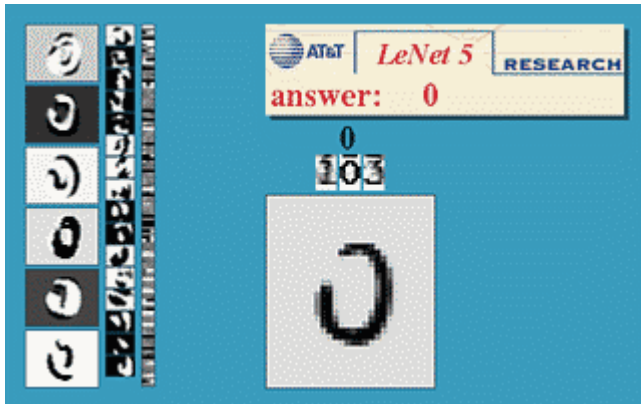
6.卷积神经网络的经典结构-LeNet



- 每个卷积层都使用 5×5 的卷积核，并在输出上使用sigmoid激活函数。
- 第一个卷积层输出通道数为6，第二个卷积层输出通道数则增加到16。
- 全连接层块含3个全连接层。它们的输出个数分别是120、84和10，其中10为输出的类别个数。
- 卷积层块的两个池化层的窗口形状均为 2×2 ，且步幅为2。

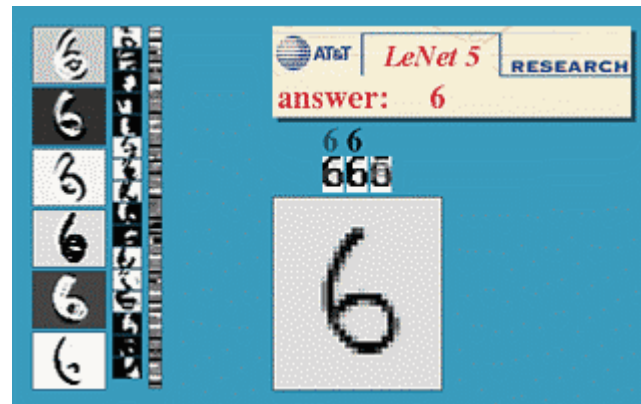
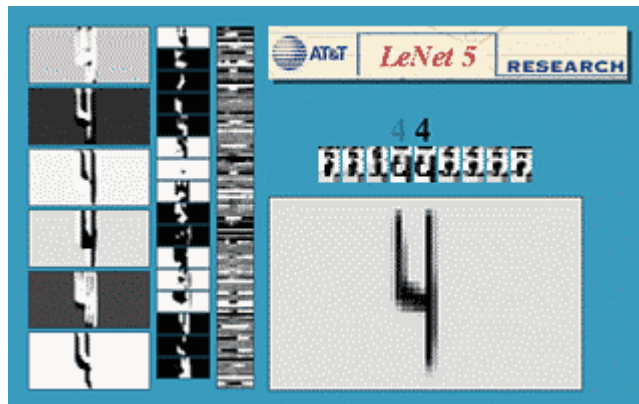
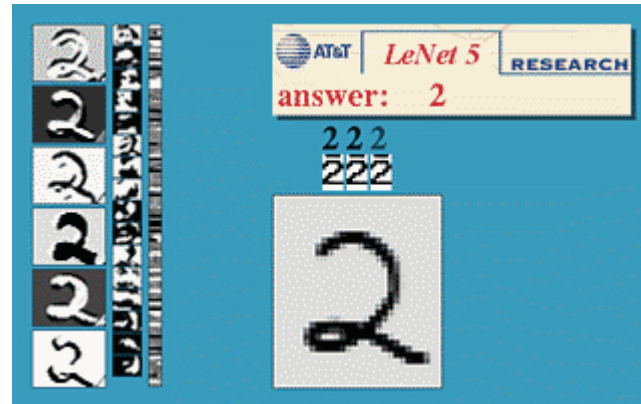
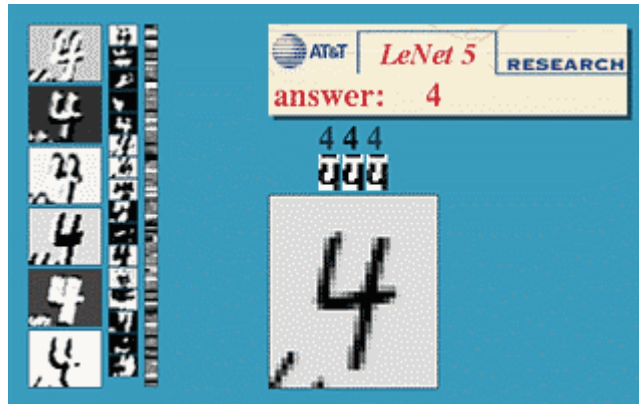
6.卷积神经网络的经典结构-LeNet

识别信件的邮政编码:



6.卷积神经网络的经典结构-LeNet

识别信件的邮政编码:



6.卷积神经网络的经典结构-LeNet

总结：

- LeNet是早期成功的神经网络，包含了现代CNN网络的基本组件。
- 卷积层保留输入形状，使图像的像素在高和宽两个方向上的相关性均可能被有效识别。
- 卷积层通过滑动窗口将同一卷积核与不同位置的输入重复计算，从而避免参数尺寸过大。

发展和限制：

- 神经网络的加速硬件没有大量普及。
- 非凸优化算法和参数初始化等理论研究还不够深入，复杂神经网络训练困难。
- 在LeNet提出后的20年里，神经网络一度被其他机器学习方法超越，如SVM。
- LeNet可以在小数据集上取得较好的成绩，但在更大的数据集上表现并不尽如人意。

6.卷积神经网络的经典结构-AlexNet

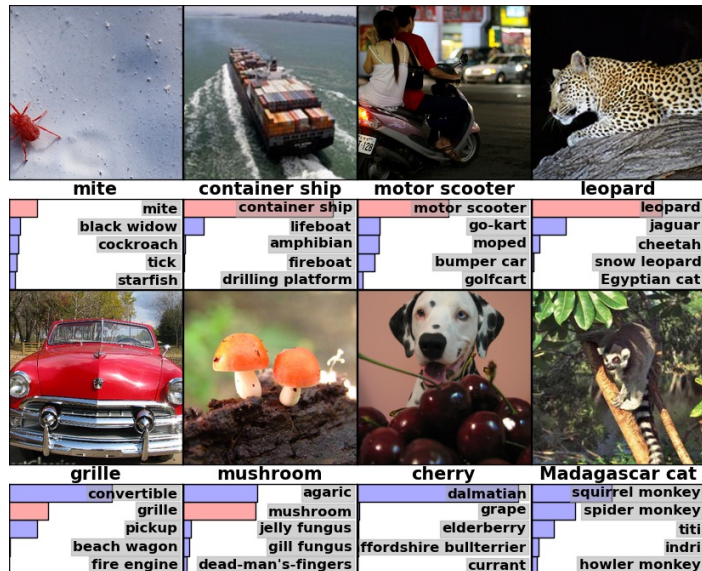
- AlexNet: 2012年被提出, 模型名字来源于论文第一作者Alex Krizhevsky, 赢得了2012 ImageNet比赛 (Top 5 误差率 15.3%, 当年第二好的模型误差率为 26.2%) 。
- AlexNet使用了8层卷积神经网络

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "Imagenet classification with deep convolutional neural networks." *Advances in neural information processing systems* 25 (2012): 1097-1105.

6.卷积神经网络的经典结构-AlexNet

背景1: ImageNet数据集

- 深度模型需要大量有标签的数据才能得到比其他经典方法更高的准确率。2010年之前, 大部分数据集只有几百至几千张图像。
- 2009年诞生的ImageNet数据集包含了超过1400万的图像, 2万多个类别, 每类有多达数千张不同的图像。
- 2010年以来, ImageNet项目每年举办一次软件比赛, 即ImageNet大规模视觉识别挑战赛 (ILSVRC) 。



ImageNet Dataset



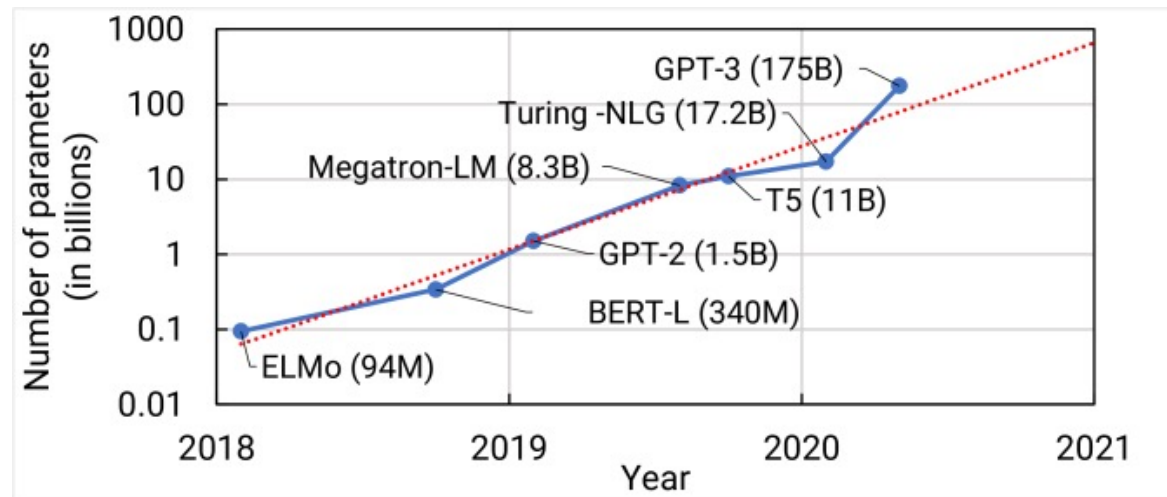
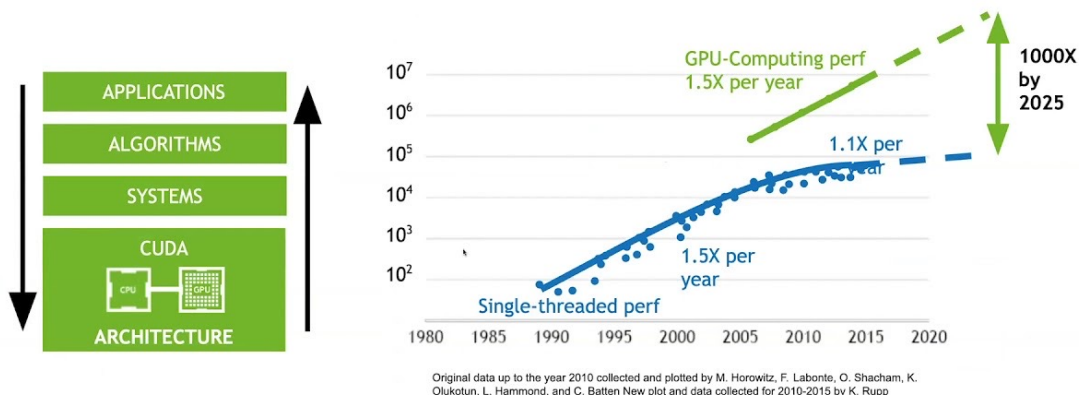
Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., ... & Fei-Fei, L. (2015). [Imagenet large scale visual recognition challenge](#). [arXiv preprint arXiv:1409.0575](#). [\[web\]](#)

6.卷积神经网络的经典结构-AlexNet

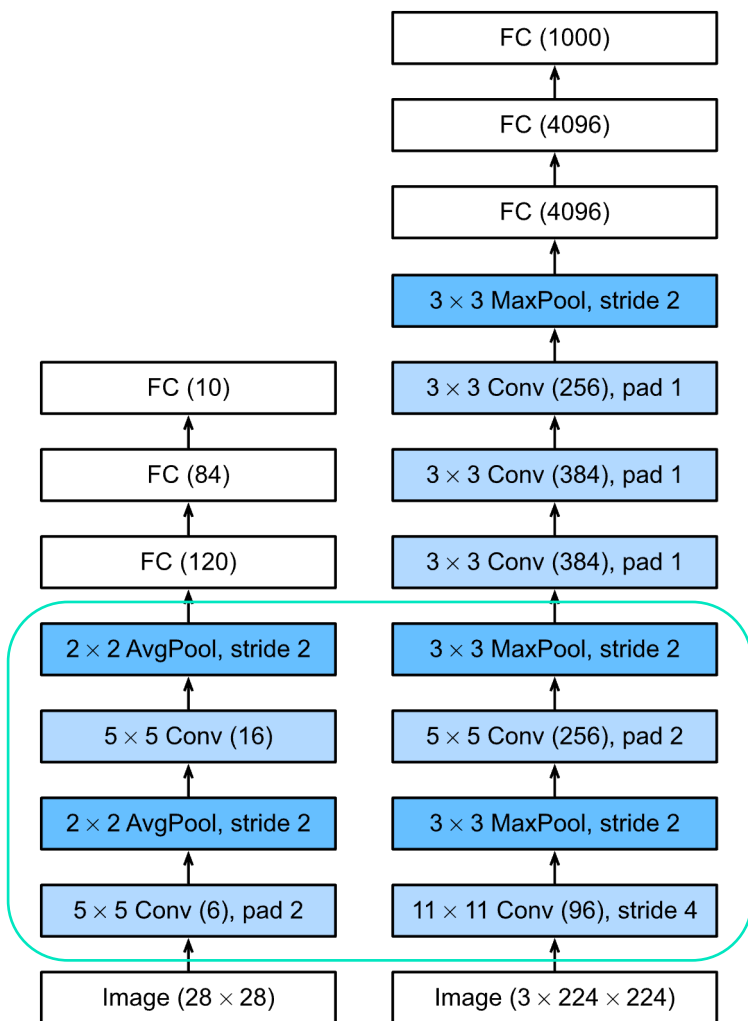
背景2：硬件水平的提升

深度学习对计算资源要求很高，GPU在2010年前后，并行计算架构CUDA等开始被用于图形计算。

RISE OF GPU COMPUTING



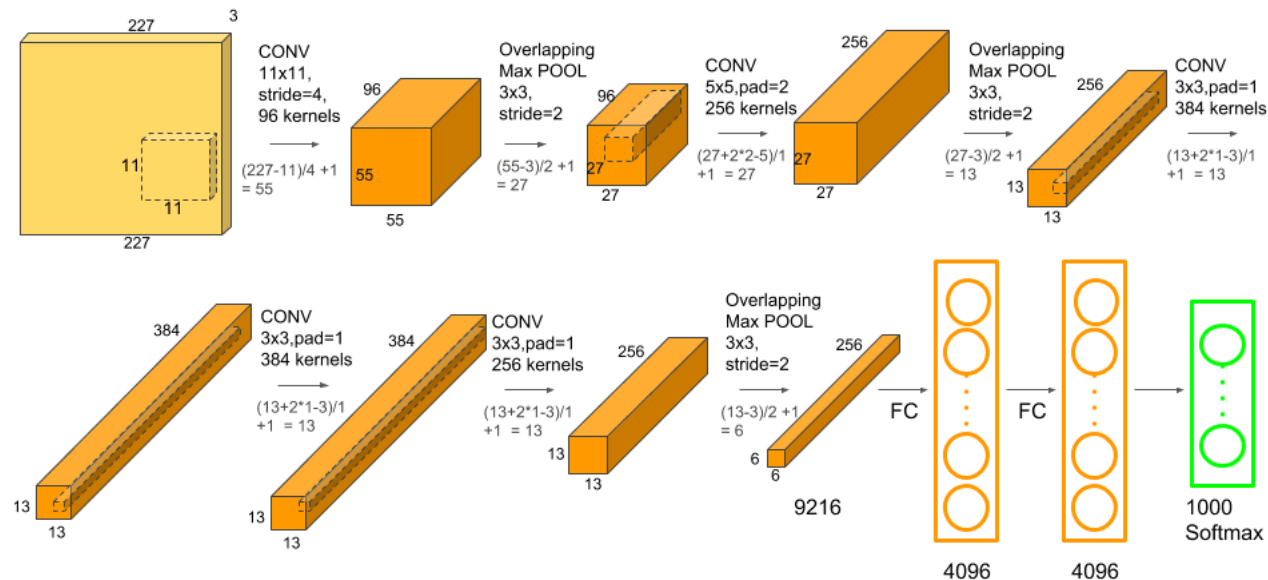
6.卷积神经网络的经典结构-AlexNet



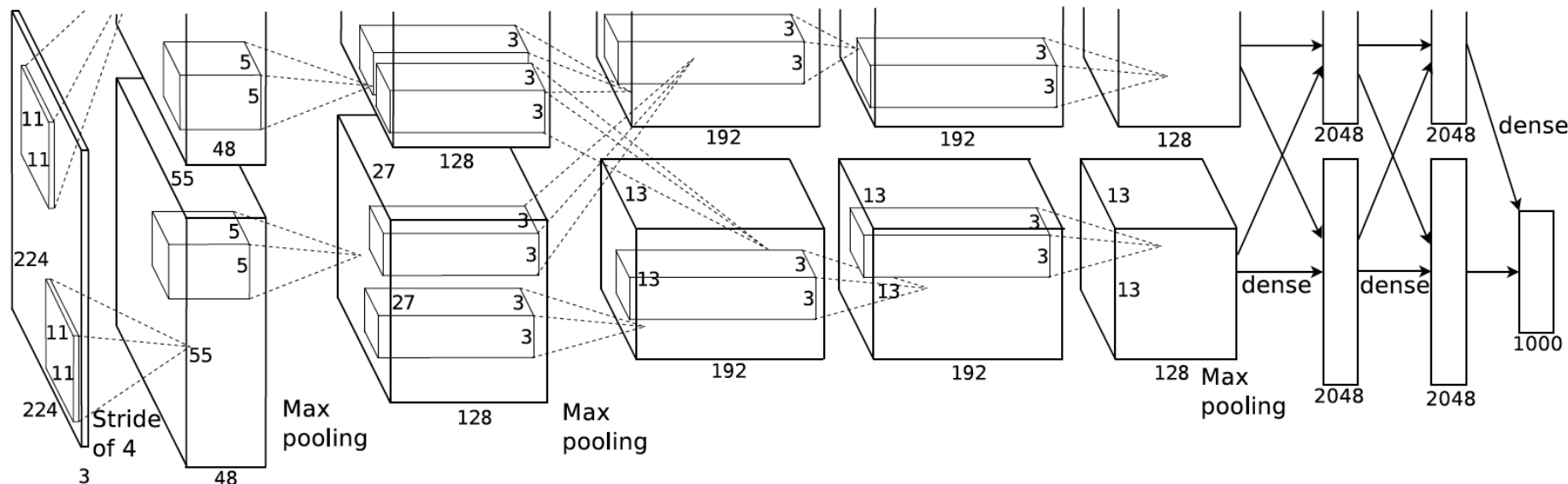
LeNet

AlexNet

更大的Kernel size
更大的Stride
Avgpool→Maxpooling/Overlap pooling
更大的Pooling窗口
更多的输出Channel

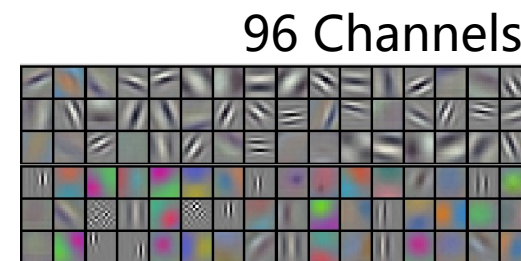
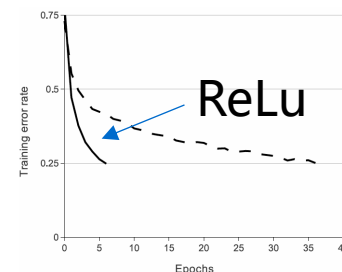


6.卷积神经网络的经典结构-AlexNet



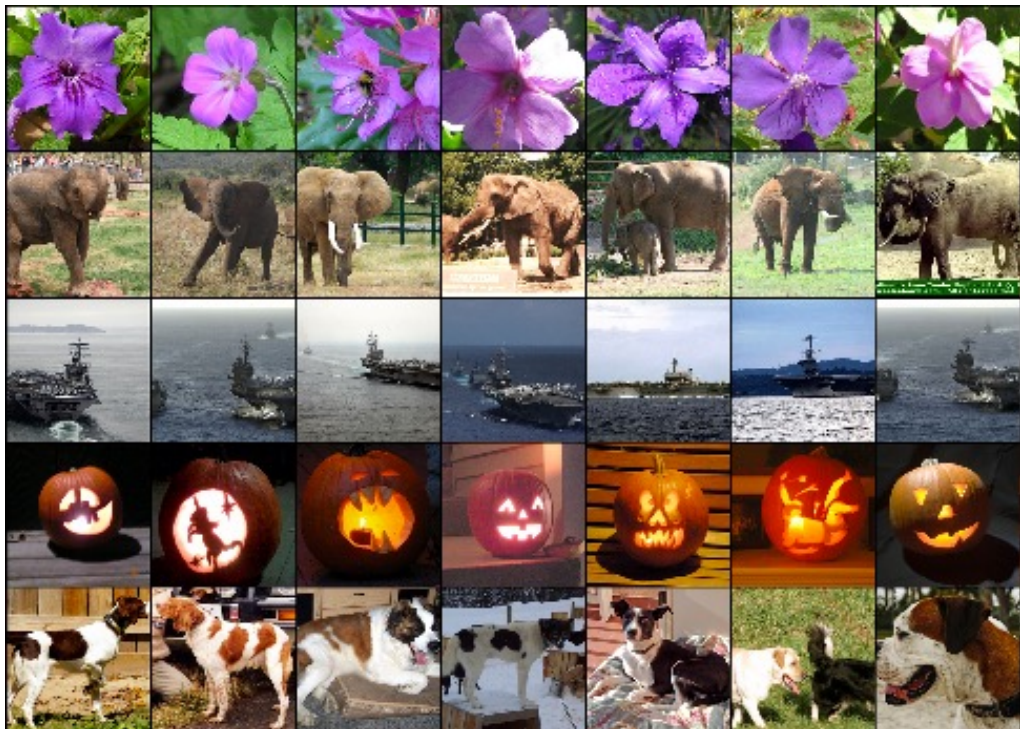
➤ 更多特点:

- AlexNet将Sigmoid激活函数改成了更加简单的ReLU激活函数。
- AlexNet通过Dropout(丢弃法) 来控制全连接层的模型复杂度。
- AlexNet引入了大量的图像增广的方法, 如翻转、裁剪和颜色变化, 进一步扩大数据集防止过拟合。
- 不需要进行图像预处理



6.卷积神经网络的经典结构-AlexNet

实验结果：



第一列中是五张ILSVRC-2010测试图像。其余各列显示了六幅训练图像，这些图像是测试图像最后一个隐藏层产生的特征向量与训练集中特征向量欧氏距离最小的图像。在像素层面上，检索到的训练图像的原图与第一列中的测试图像的原图的欧式距离一般不接近。

Model	Top-1	Top-5
<i>Sparse coding [2]</i>	47.1%	28.2%
<i>SIFT + FVs [24]</i>	45.7%	25.7%
CNN	37.5%	17.0%

Table 1: Comparison of results on ILSVRC-2010 test set. In *italics* are best results achieved by others.

Model	Top-1 (val)	Top-5 (val)	Top-5 (test)
<i>SIFT + FVs [7]</i>	—	—	26.2%
1 CNN	40.7%	18.2%	—
5 CNNs	38.1%	16.4%	16.4%
1 CNN*	39.0%	16.6%	—
7 CNNs*	36.7%	15.4%	15.3%

Table 2: Comparison of error rates on ILSVRC-2012 validation and test sets. In *italics* are best results achieved by others. Models with an

6.卷积神经网络的经典结构-AlexNet

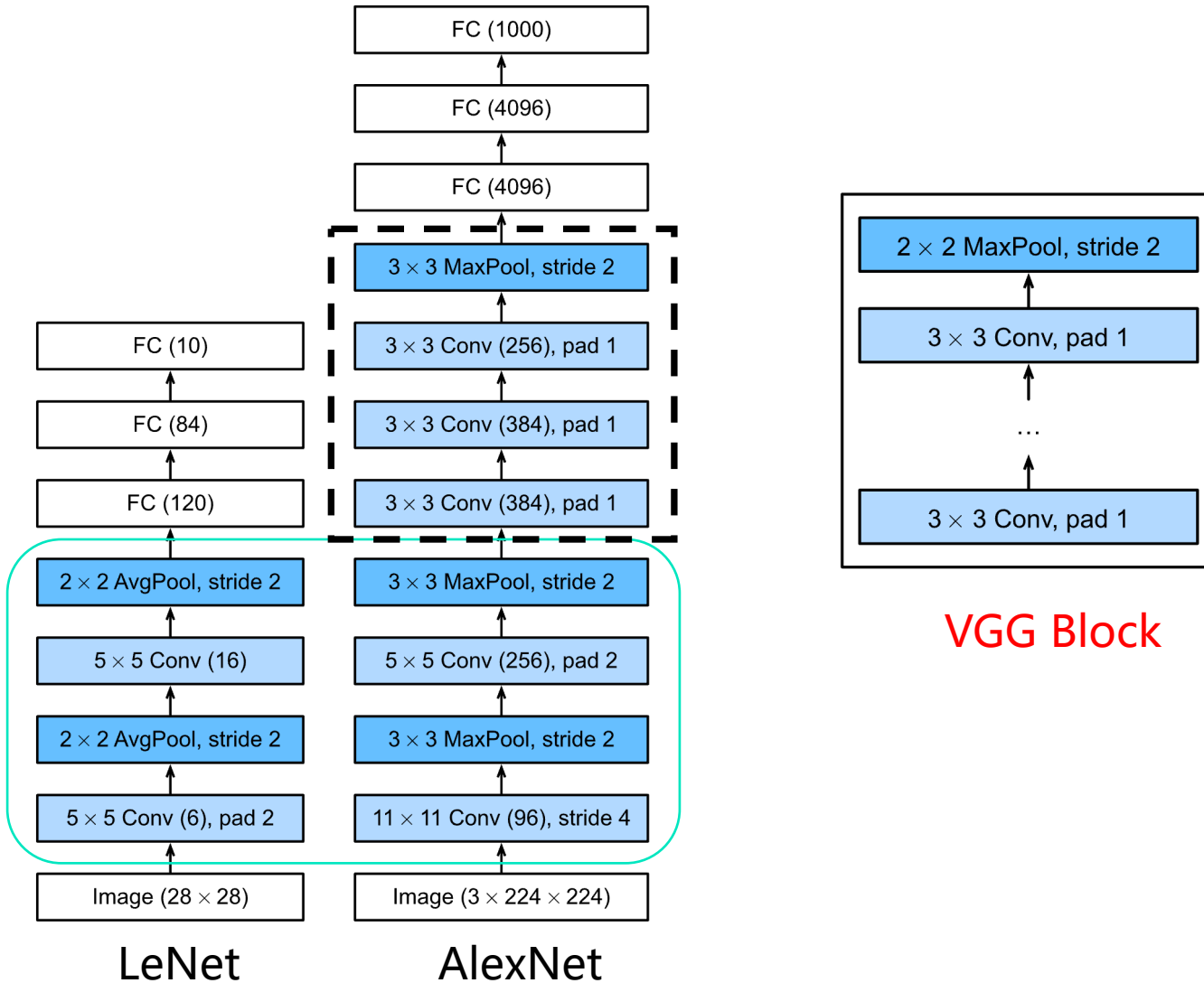
总结:

- AlexNet不仅比LeNet的神经网络层数更多更深, 使用了更多的卷积层和更大的参数空间来拟合大规模数据集ImageNet, 是浅层神经网络和深度神经网络的分界线。
- 可以学习更复杂的图像高维特征。

缺点:

- AlexNet比Lenet更大更深, 表明深度卷积神经网络可以取得出色的结果。但并没有提供简单的规则指导如何变得更大更深来提升性能

6.卷积神经网络的经典结构-VGG



➤ VGG组成规律:

连续使用数个相同的填充为1、窗口形状为3x3的卷积层后接上一个步幅为2、窗口形状为2x2的最大池化层。卷积层保持输入的高和宽不变，而池化层则对其减半。

6.卷积神经网络的经典结构-VGG

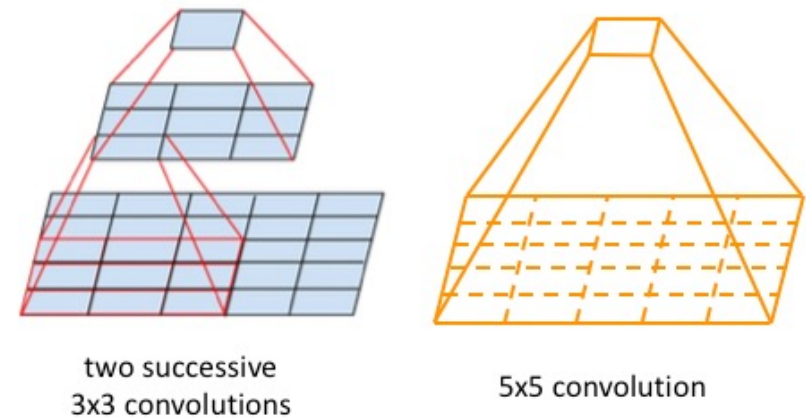
- VGG的名字来源于作者所在的**V**isual **G**eometry **G**roup实验室。
- 该网络是在ILSVRC 2014上的相关工作，主要工作是证明了增加网络的深度能够在一定程度上影响网络最终的性能。
- VGG有两种结构，分别是VGG16和VGG19，两者并没有本质上的区别，只是网络深度不一样。
- 提出了可以通过重复使用简单的基础块来构建深度模型的思路。

Simonyan, Karen, and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition." *arXiv preprint arXiv:1409.1556* (2014).

6.卷积神经网络的经典结构-VGG

原理：在VGG中使用3个3x3卷积核代替7x7(ZFNet)卷积核，使用2个3x3卷积核来代替5x5 (LeNet) 卷积核，在保证具有相同感知野的条件下，提升了网络的深度，减小了网络参数，在一定程度上提升了神经网络的效果。

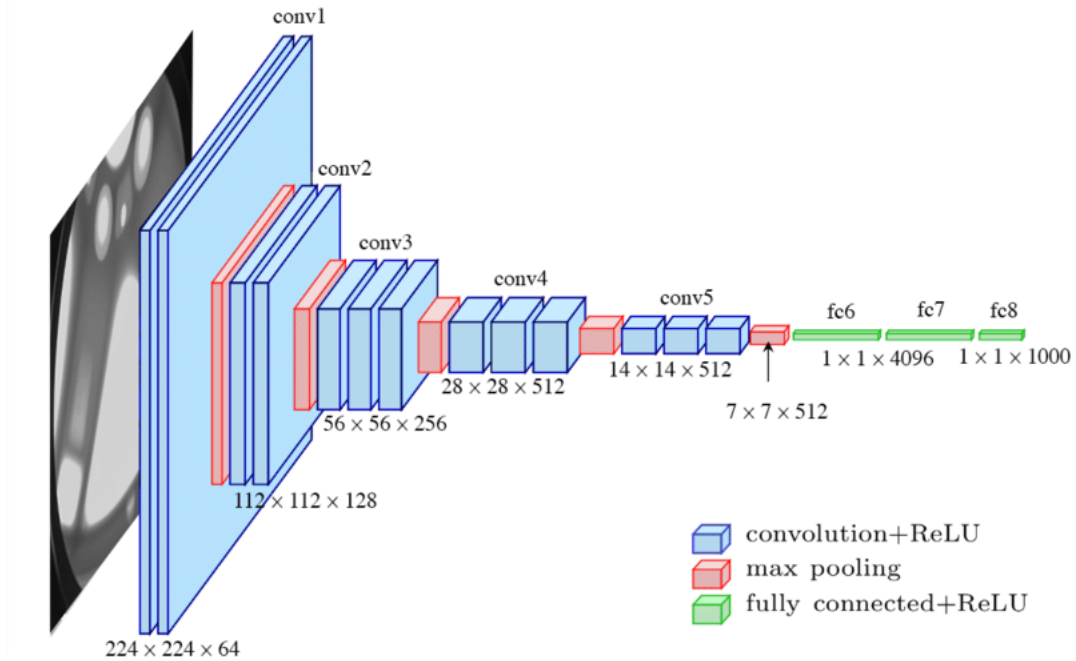
2个3x3卷积核代替5 x 5卷积核
具有相同感知野的条件下
(计算量差不多)



6.卷积神经网络的经典结构-VGG

VGG架构：多个VGG块堆叠后接全连接层

不同次数的重复得到不同的架构



LeNet: 卷积+池化+全连接

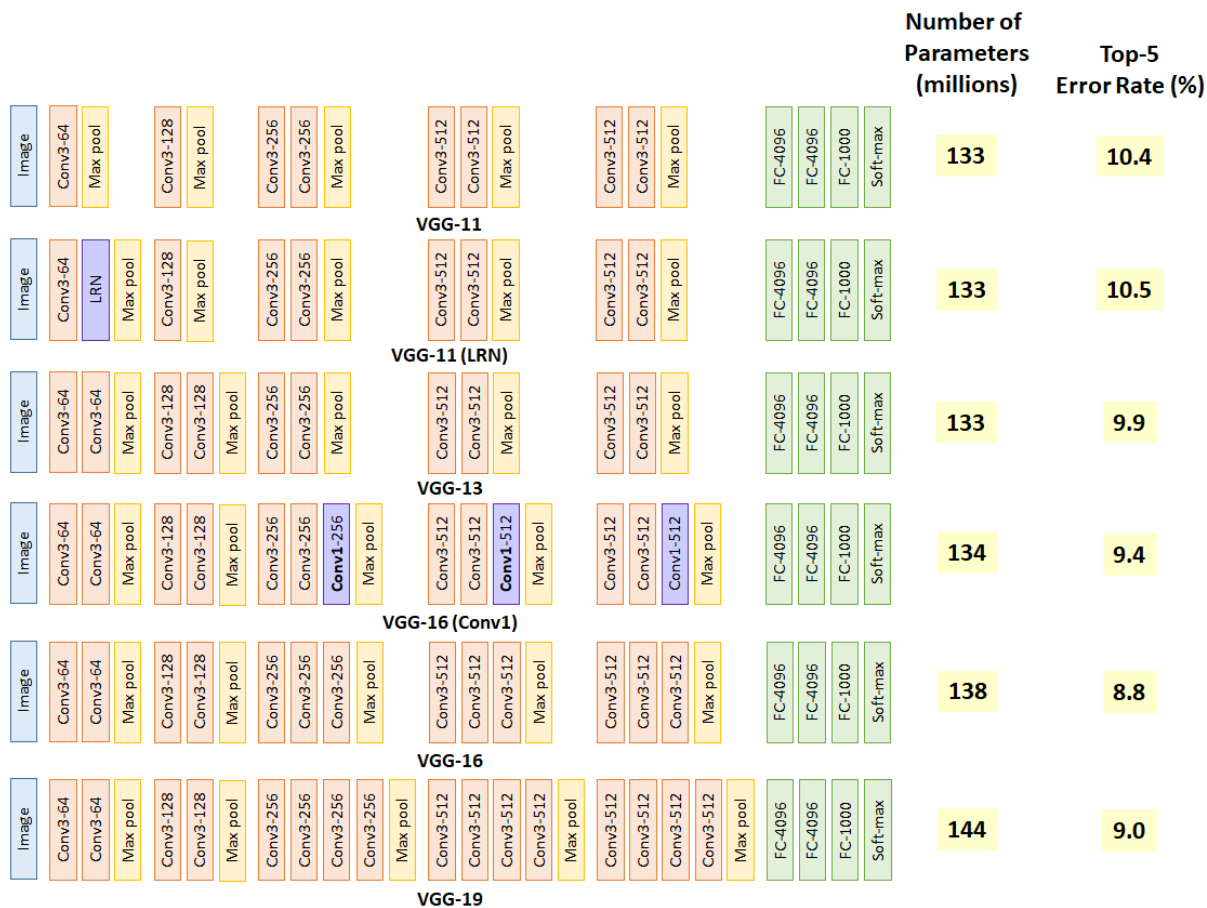


AlexNet: ReLU+Dropout+Maxpooling



VGG: 使用简单的基础模块构建深度模型

6.卷积神经网络的经典结构-VGG



ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

6.卷积神经网络的经典结构-VGG

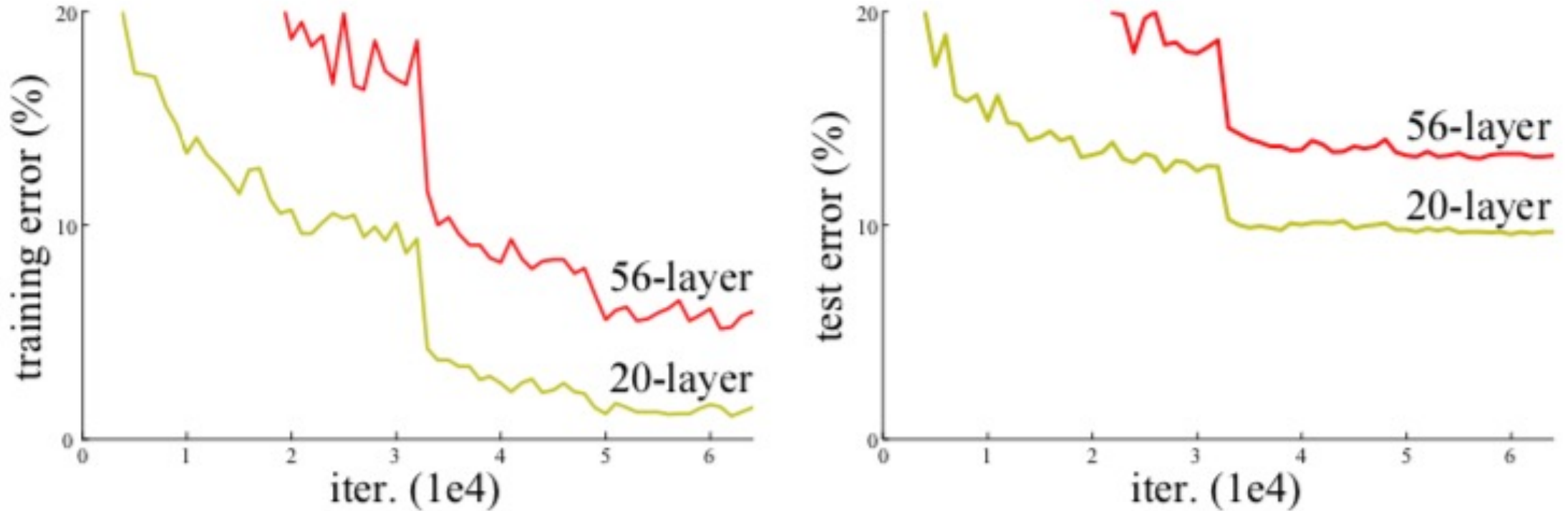
总结：

- VGG结构简洁，整个网络使用了同样大小的卷积核（3x3）和最大池化尺寸（2x2）。
- 证明了小滤波器（3x3）卷积层组合比一个大滤波器（5x5或7x7）卷积层效果好。
- 验证了不断加深网络结构可以提升性能。

思考：

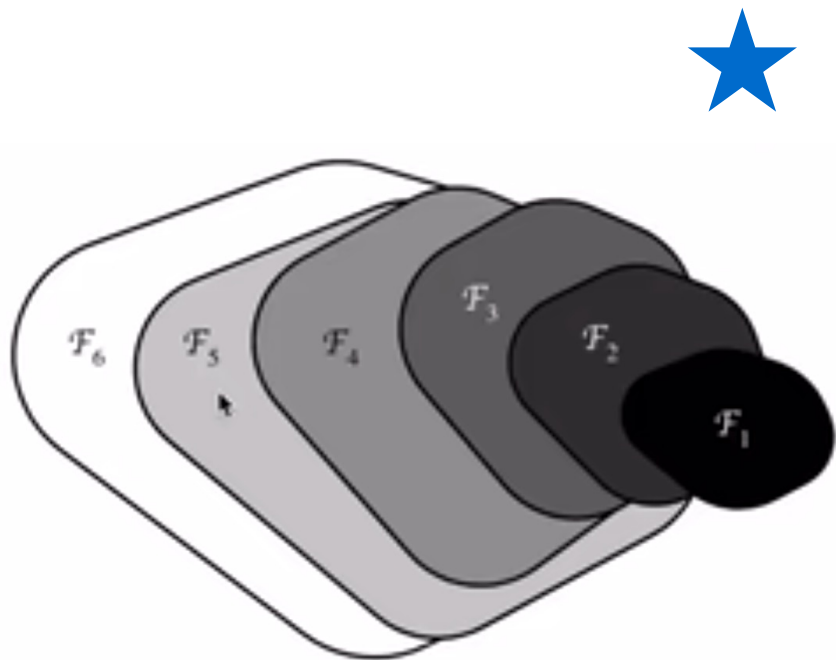
- 对神经网络模型添加新的层，充分训练后的模型是否能更有效的降低误差？原理上可行，但实践中添加过多的层后训练误差往往不升反降。

6.卷积神经网络的经典结构-ResNet

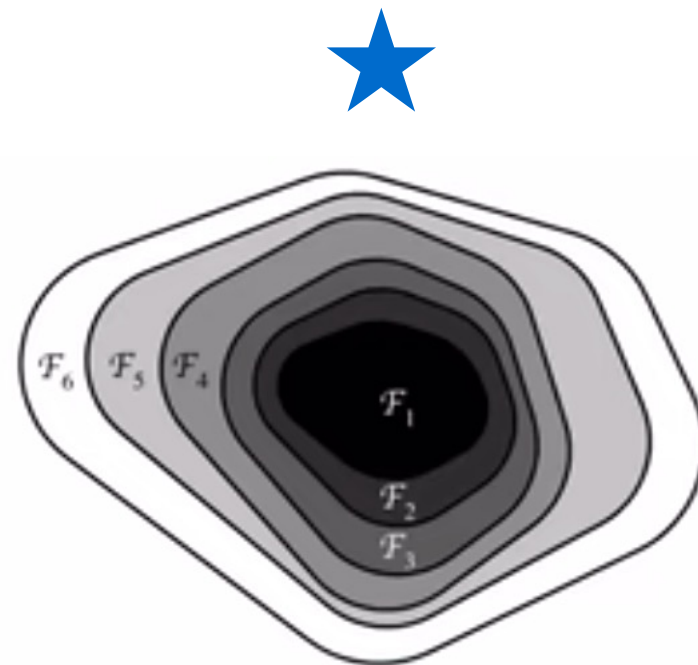


深度神经网络的“退化”问题

6.卷积神经网络的经典结构-ResNet



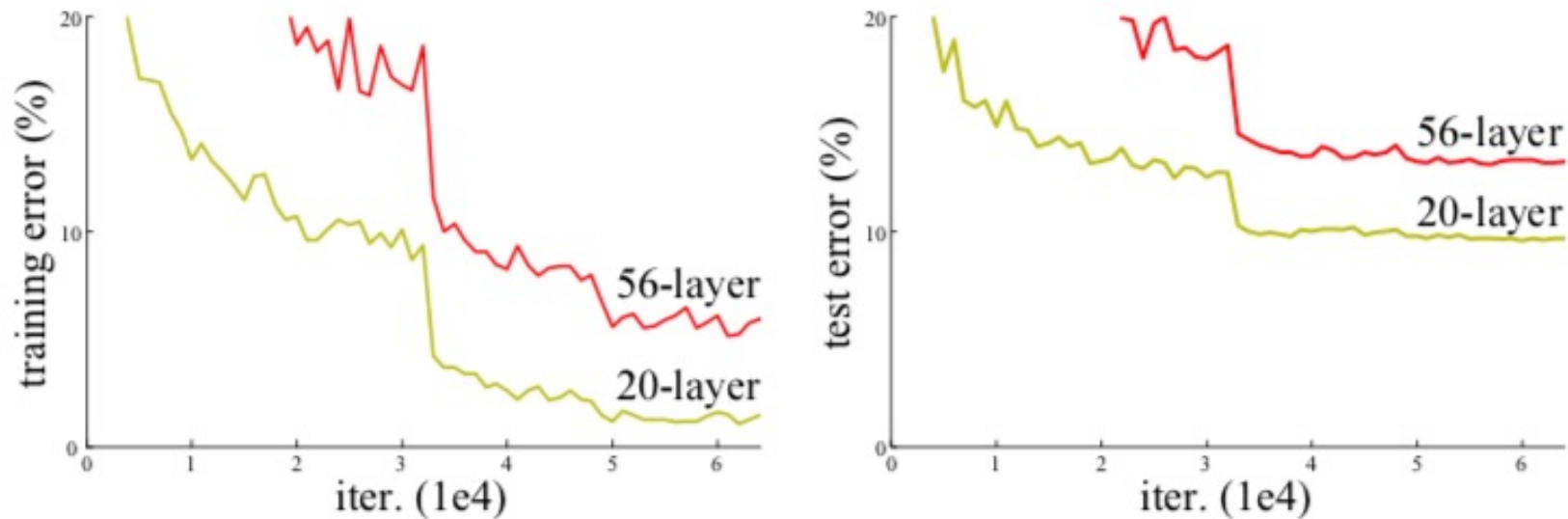
非嵌套的函数集



嵌套的函数集

6.卷积神经网络的经典结构-ResNet

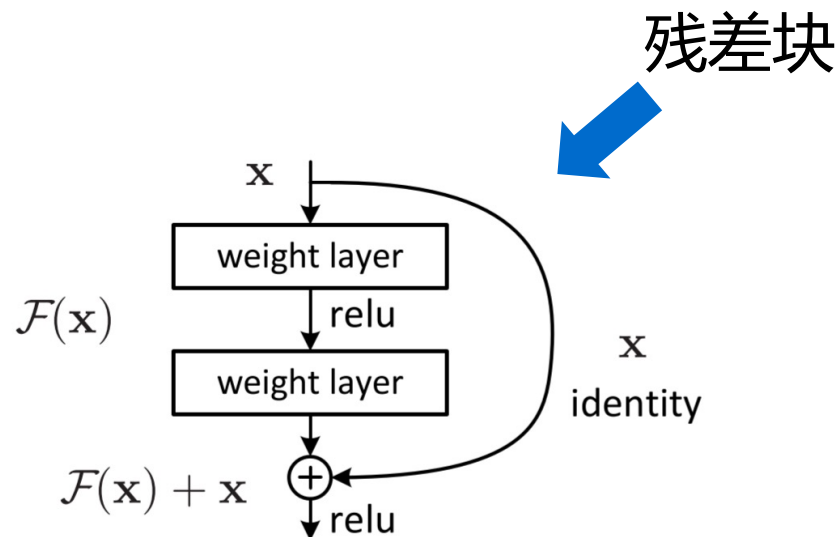
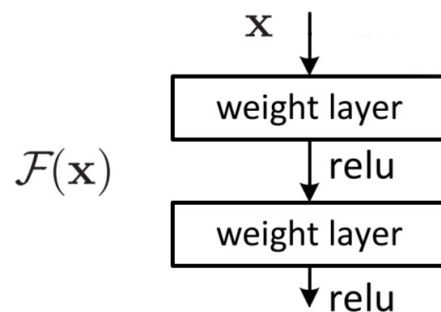
- 2015年，何凯明等人提出的残差网络（ResNet）在ImageNet图像识别大赛夺冠，并深刻影响了后来的深度学习网络的设计。
- ResNets解决深度神经网络的“退化”问题。



He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770-778).

6.卷积神经网络的经典结构-ResNet

残差链接：



- 设输入为 x ，如果期望的潜在映射为 $H(x)$ ，可以拟合的函数为 $F(x)$ 。与其让 $F(x)$ 直接学习潜在的映射，不如去学习残差 $H(x)-x$ ，即 $F(x):=H(x)-x$ ，这样原本的前向路径上就变成了 $F(x)+x$ ，用 $F(x)+x$ 来拟合 $H(x)$ 。
- 残差映射在实际中更容易优化。
- 残差块中的输入可以通过跨层的数据线（短路链接，Shortcut）更快的向前传播。

6.卷积神经网络的经典结构-ResNet

Identity Mapping:

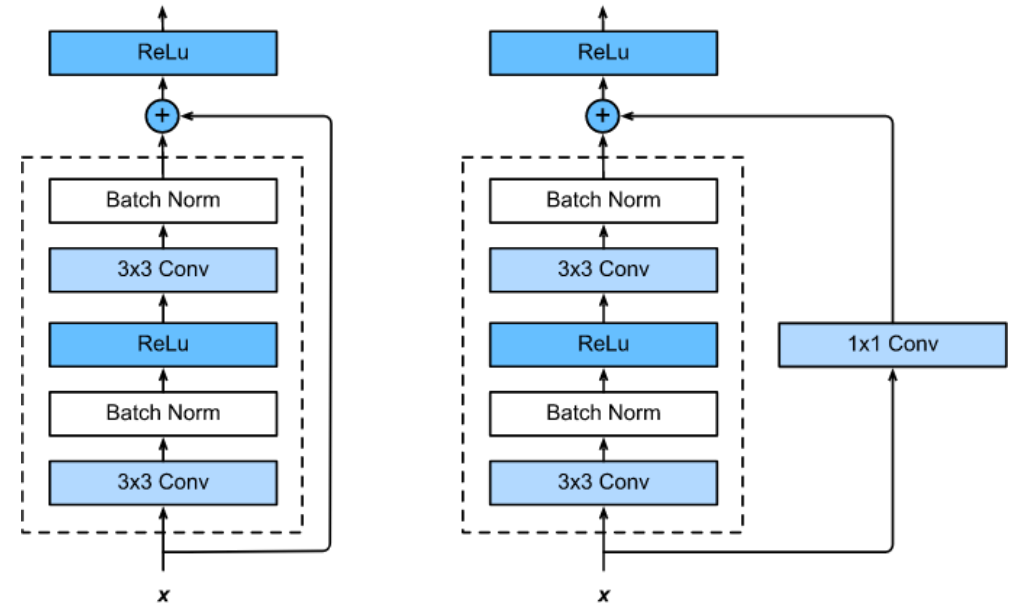
$$y = \mathcal{F}(x, \{W_i\}) + x$$

$\mathcal{F}$ 与 x 直接element wise相加，最后 y 加入一个激活函数，没有引入任何新的参数，也没有增加计算复杂度。

$\mathcal{F}$ 与 x 维度（输出通道）不一样怎么办？

- 1) 直接对 x 补0。
- 2) 增加一个网络层（参数 W_s ），改变 x 的维度。

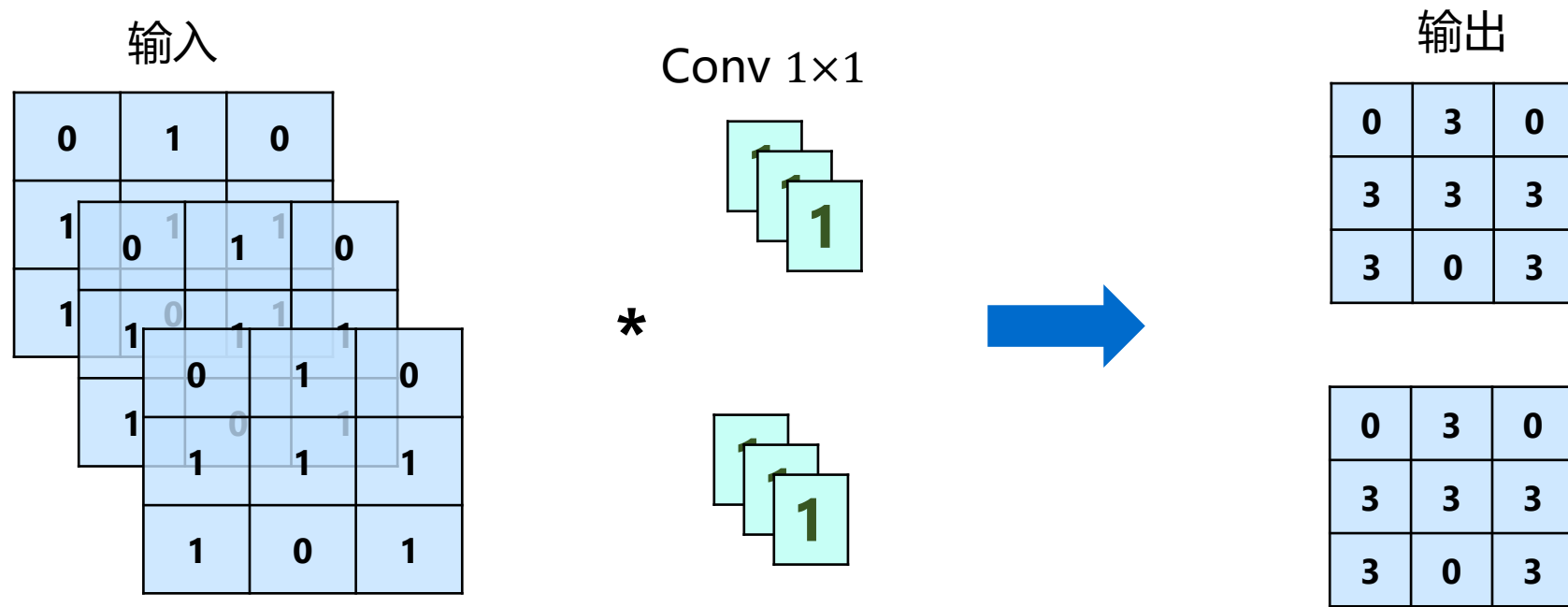
$$y = \mathcal{F}(x, \{W_i\}) + W_s x$$



6.卷积神经网络的工作原理



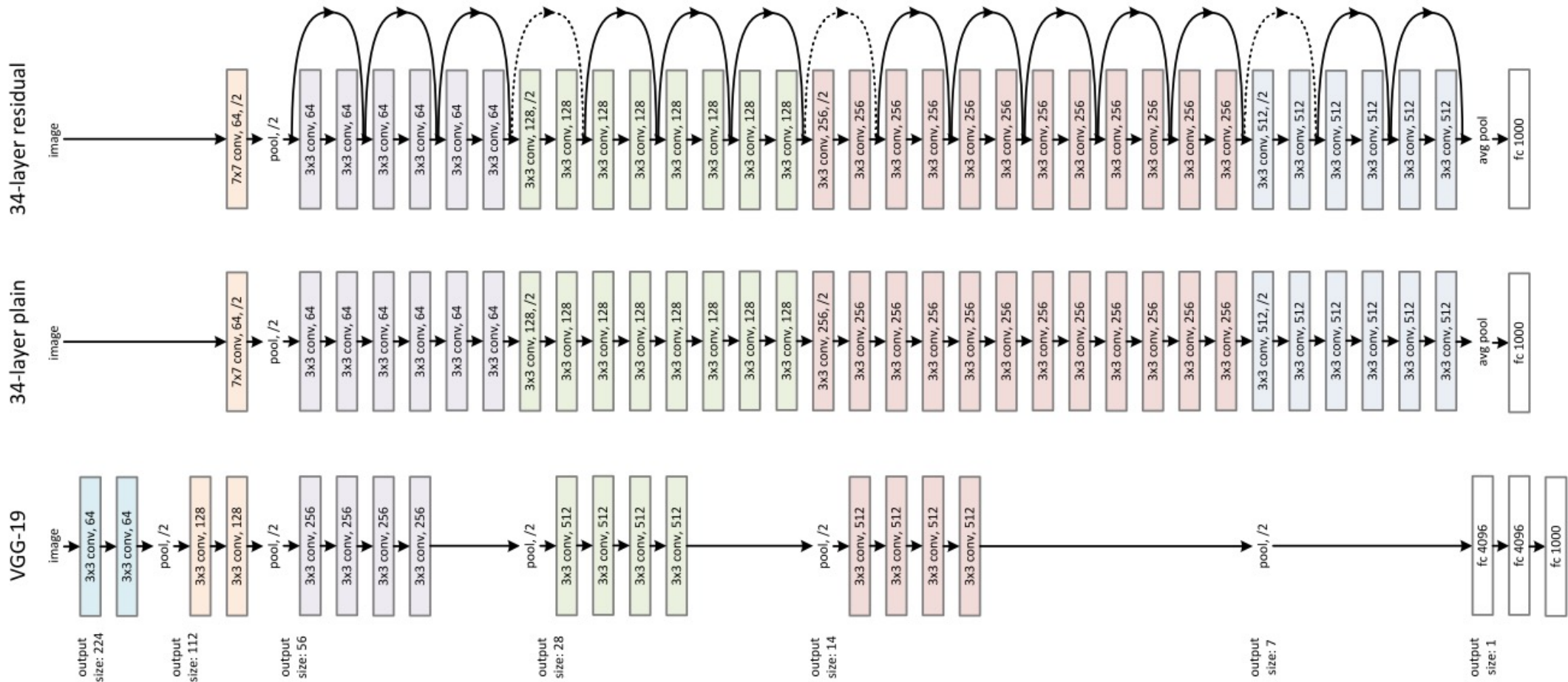
➤ 补充：通道 (Channel)---Kernel size:1*1的卷积



1×1卷积层被当做保持高和宽维度形状不变的全连接层使用，可以通过调整网络层之间的通道数来控制模型复杂度。

6.卷积神经网络的经典结构-ResNet

网络结构：



6.卷积神经网络的经典结构-ResNet

实验结果：

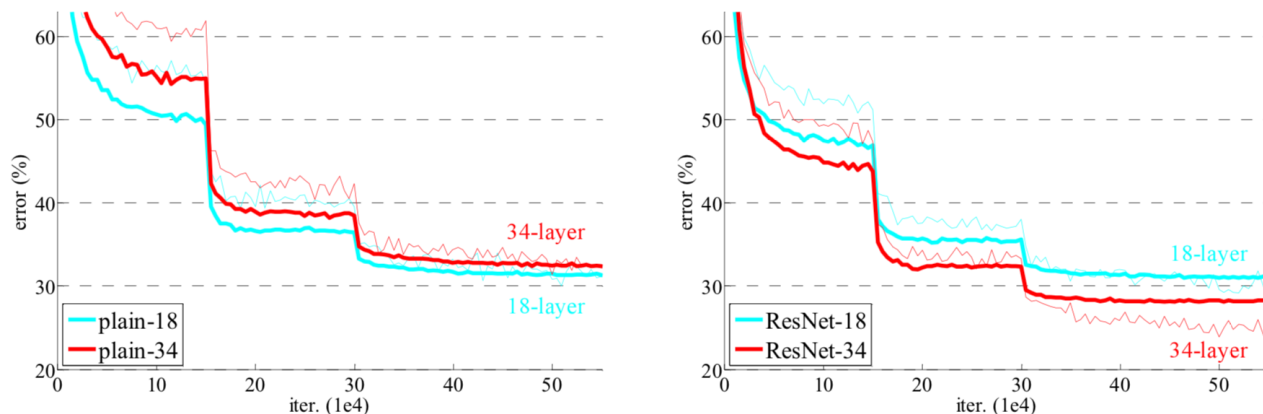


Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	25.03

Table 2. Top-1 error (% , 10-crop testing) on ImageNet validation. Here the ResNets have no extra parameter compared to their plain counterparts. Fig. 4 shows the training procedures.

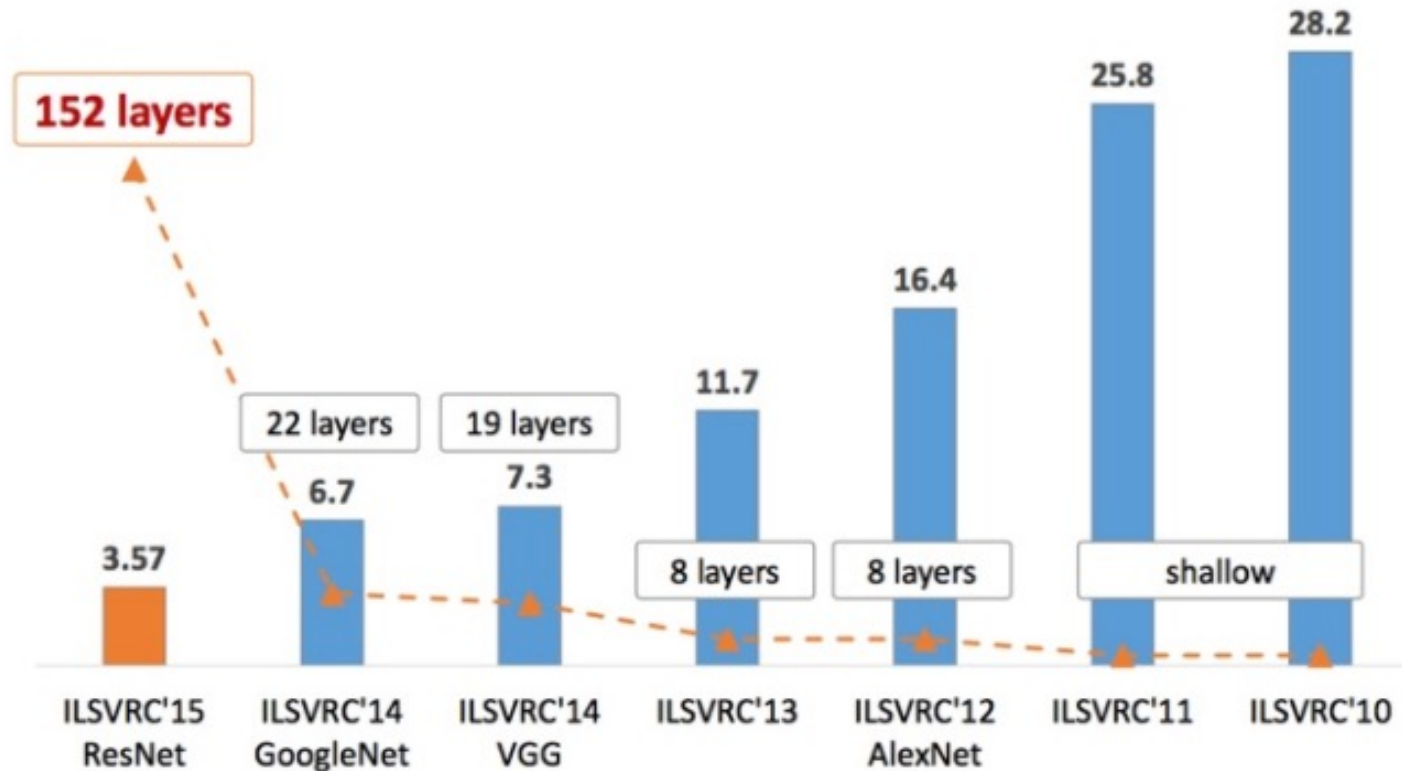
ResNet解决 “深度增加带来的退化问题”

- Train error和test error都降低了，证明了网络深度确实可以帮助提升网络的性能。degradation problem在一定程度上得到了解决。
- 相对于Plain 34-layers，ResNet 34的 top-1 error rate也降低了3.5%。Resnet实现了在没有增加任何参数的情况下，获得了更低error rate，网络更加高效。
- 从Plain/Residual 18比较两者的error rate差不多，但是ResNet却能够收敛得更快。

6.卷积神经网络的经典结构-ResNet

总结：

- ResNet采用残差块和Shortcut identity mapping，取得了极其好的效果。



- 掌握卷积神经网络中卷积层和池化层的工作原理
- 掌握填充、步幅、通道等定义。
- 了解近年来具有代表性的深度卷积神经网络的设计思路